

Lab Manual

AIC 380 – Artificial Neural Networks



(1st Draft Version)

CUI

**Department of Computer Science
Islamabad Campus**

Lab Contents:

This lab manual provides an understanding of implementing artificial neural networks in python. The topics include: implementation of perceptron; artificial neural networks; self-organizing maps; learning vector quantizations;; Convolutional Neural Networks; Recurrent Neural Networks; long-short term memory.

Student Outcomes (SO)

S.#	Description
2	Overview of Neural Networks: Definition, History, Biological Neurons, Models of Neurons, Artificial & Biological Neural Networks, and Neural Network Models
3	Single Layer Perceptrons: Least Mean Square Algorithm, Learning Curves, Learning Rates, Perceptron; Multilayer Perceptrons: XOR Problem, Backpropagation Algorithm, Heuristic for Improving the Backpropagation Algorithm; and Adaline & Madaline Networks.
4	Pattern Association: Hetroassociative Memory Neural Network, Autoassociative Neural Network, Iterative Autoassociative Net; Bidirectional Associative Memory (BAM) & Hopfield Memory: Architecture, Algorithm, Analysis, and Applications
5	Radial-Basis Function Networks: Interpolation, and Regularization & Learning strategies.
6	Kohonen Self-Organizing Maps: SOM Algorithm, Learning Vector Quantization; Adaptive Resonance Theory (ART); Spatiotemporal Networks (STNS); and Neocognitron.
7	Convolutional Neural Networks (CNN): Motivation, Convolutional layers, Additional layers, Residual Nets; Recurrent Neural Networks (RNN): Motivation, Sequential Processing, Stability, and Gated Nets (LSTM, GRU).

Intended Learning Outcomes

Sr.#	Description	Blooms Taxonomy Learning Level	SO
CLO -2	Apply appropriate Neural Networks to solve practical	<i>Applying</i>	2-4
CLO -3	Apply deep learning techniques for classification and recognition problems.	<i>Applying</i>	2-4
CLO-4	Implement appropriate Neural Networks to solve practical problems.	<i>Applying</i>	2-4

Lab Assessment Policy

The lab work done by the student is evaluated using Psycho-motor rubrics defined by the course instructor, viva-voce, project work/performance. Marks distribution is as follows:

Assignments	Lab Mid Term Exam	Lab Terminal Exam	Total
25	25	50	100

Note: Midterm and Final term exams must be computer based.

List of Labs

Lab #	Main Topic	Page #
Lab 01	Basics of Python	4
Lab 02	Introduction of numpy	22
Lab 03	Implementation and training of perceptron using learning rule in python	36
Lab 06	Implementation of Self-organizing map in python	47
Lab 07	Implementation of learning vector quantization in python	56
Lab 08		
Lab 09	Mid Term Exam	
Lab 10	Introduction to Keras and implementation of ANN in Keras	63
Lab 11	Implementation of convolution neural network (CNNs) in Keras	71
Lab 12	Implementation of recurrent neural networks (RNNs) in Keras	82
Lab 13	Implementation of long-short term memory (LSTM) and gated recurrent units (GRUs) in Keras	89
Lab 14	Working on project	
Lab 15	Final Term Exam	

Lab 01

Objective:

The objective of this lab is to give students a recap of python basics important for understanding the implementation of artificial neural networks in python.

Activity Outcomes:

The activities provide hands - on practice with the following topics

- Implement a data type in python.
- Implement flow control statement.
- Implement functions and classes.
- Implement file IO.

Instructor Note:

As pre-lab activity, read relevant chapters from Madhavan, Samir. Mastering python for data science. Packt Publishing Ltd, 2015: <https://github.com/AmandaZou/Data-Science-books-/blob/master/Mastering%20Python%20for%20Data%20Science.pdf>

1) Useful Concepts

1. **Syntax:** Python has no mandatory statement termination characters and blocks are specified by indentation. Indent to begin a block, dedent to end one. Statements that expect an indentation level end in a colon (:). Comments start with the pound (#) sign and are single-line, multi-line strings are used for multi-line comments. Values are assigned (in fact, objects are bound to names) with the equals sign (“=”), and equality testing is done using two equals signs (“==”). You can increment/decrement values using the += and -= operators respectively by the right-hand amount. This works on many datatypes, strings included. You can also use multiple variables on one line. For example:

```
>>> myvar = 3
>>> myvar += 2
>>> myvar
5
>>> myvar -= 1
>>> myvar
4
"""This is a multiline comment.
The following lines concatenate the two strings."""
>>> mystring = "Hello"
>>> mystring += " world."
>>> print(mystring)
Hello world.
# This swaps the variables in one line(!).
# It doesn't violate strong typing because values aren't
# actually being assigned, but new objects are bound to
# the old names.
>>> myvar, mystring = mystring, myvar
```

2. **Data types:** The data structures available in python are lists, tuples and dictionaries. Sets are available in the sets library (but are built-in in Python 2.5 and later). Lists are like one-dimensional arrays (but you can also have lists of other lists), dictionaries are associative arrays (a.k.a. hash tables) and tuples are immutable one-dimensional arrays (Python “arrays” can be of any type, so you can mix e.g. integers, strings, etc in lists/dictionaries/tuples). The index of the

first item in all array types is 0. Negative numbers count from the end towards the beginning, -1 is the last item. Variables can point to functions. The usage is as follows

```
>>> sample = [1, ["another", "list"], ("a", "tuple")]
>>> mylist = ["List item 1", 2, 3.14]
>>> mylist[0] = "List item 1 again" # We're changing the item.
>>> mylist[-1] = 3.21 # Here, we refer to the last item.
>>> mydict = {"Key 1": "Value 1", 2: 3, "pi": 3.14}
>>> mydict["pi"] = 3.15 # This is how you change dictionary values
>>> mytuple = (1, 2, 3)
>>> myfunction = len
>>> print(myfunction(mylist))
```

You can access array ranges using a colon (:). Leaving the start index empty assumes the first item, leaving the end index assumes the last item. Indexing is inclusive-exclusive, so specifying [2:10] will return items [2] (the third item, because of 0-indexing) to [9] (the tenth item), inclusive (8 items). Negative indexes count from the last item backwards (thus -1 is the last item) like so:

```
>>> mylist = ["List item 1", 2, 3.14]
>>> print(mylist[:])
['List item 1', 2, 3.1400000000000001]
>>> print(mylist[0:2])
['List item 1', 2]
>>> print(mylist[-3:-1])
['List item 1', 2]
>>> print(mylist[1:])
[2, 3.14]
# Adding a third parameter, "step" will have Python step in
# N item increments, rather than 1.
# E.g., this will return the first item, then go to the third and
# return that (so, items 0 and 2 in 0-indexing).
>>> print(mylist[::2])
['List item 1', 3.14]
```

3. **Strings:** Its strings can use either single or double quotation marks, and you can have quotation marks of one kind inside a string that uses the other kind (i.e. "He said 'hello'." is valid). Multiline strings are enclosed in _triple double (or single) quotes_ ("""). Python strings are always Unicode, but there is another string type that is pure bytes. Those are called bytestrings and are represented with the b prefix, for example b'Hello \x01'. . To fill a string with values, you use the % (modulo) operator and a tuple. Each %s gets replaced with an item from the tuple, left to right, and you can also use dictionary substitutions, like so:

```
>>> print("Name: %s\
Number: %s\
String: %s" % (myclass.name, 3, 3 * "-"))
Name: Stavros
Number: 3
String: ---

strString = """This is
a multiline
string."""

# WARNING: Watch out for the trailing s in "%(key)s".
>>> print("This %(verb)s a %(noun)s." % {"noun": "test", "verb": "is"})
This is a test.

>>> name = "Stavros"
>>> "Hello, {}".format(name)
Hello, Stavros!
>>> print(f"Hello, {name}!")
Hello, Stavros!
```

4. **Flow control statements:** Flow control statements are if, for, and while. There is no switch; instead, use if. Use for to enumerate through members of a list. To obtain a sequence of numbers you can iterate over, use range(<number>). These statements' syntax is thus:

```
>>> print(range(10))
range(0, 10)
>>> rangelist = list(range(10))
>>> print(rangelist)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

for number in range(10):
    # Check if number is one of
    # the numbers in the tuple.
    if number in (3, 4, 7, 9):
        # "Break" terminates a for without
        # executing the "else" clause.
        break
    else:
        # "Continue" starts the next iteration
        # of the loop. It's rather useless here,
        # as it's the last statement of the loop.
        continue
else:
    # The "else" clause is optional and is
    # executed only if the loop didn't "break".
    pass # Do nothing

if rangelist[1] == 2:
    print("The second item (lists are 0-based) is 2")
elif rangelist[1] == 3:
    print("The second item (lists are 0-based) is 3")
else:
    print("Dunno")

while rangelist[1] == 1:
    print("We are trapped in an infinite loop!")
```

5. **Functions:** Functions are declared with the `def` keyword. Optional arguments are set in the function declaration after the mandatory arguments by being assigned a default value. For named arguments, the name of the argument is assigned a value. Functions can return a tuple (and using tuple unpacking you can effectively return multiple values). Lambda functions are ad hoc functions that are comprised of a single statement. Parameters are passed by reference, but immutable types (tuples, ints, strings, etc) cannot be changed in the caller by the callee. This is

because only the memory location of the item is passed, and binding another object to a variable discards the old one, so immutable types are replaced. For example:

```
# Same as def funcvar(x): return x + 1
funcvar = lambda x: x + 1
>>> print(funcvar(1))
2

# an_int and a_string are optional, they have default values
# if one is not passed (2 and "A default string", respectively).
def passing_example(a_list, an_int=2, a_string="A default string"):
    a_list.append("A new item")
    an_int = 4
    return a_list, an_int, a_string

>>> my_list = [1, 2, 3]
>>> my_int = 10
>>> print(passing_example(my_list, my_int))
([1, 2, 3, 'A new item'], 4, "A default string")
>>> my_list
[1, 2, 3, 'A new item']
>>> my_int
10
```

6. **Classes:** Python supports a limited form of multiple inheritance in classes. Private variables and methods can be declared (by convention, this is not enforced by the language) by adding a leading underscore (e.g. `_spam`). We can also bind arbitrary names to class instances. An example follows:

```
class MyClass(object):
    common = 10
    def __init__(self):
        self.myvariable = 3
    def myfunction(self, arg1, arg2):
        return self.myvariable

# This is the class instantiation

>>> classinstance = MyClass()
>>> classinstance.myfunction(1, 2)
3
# This variable is shared by all instances.
>>> classinstance2 = MyClass()
>>> classinstance.common
10
>>> classinstance2.common
10
# Note how we use the class name
# instead of the instance.
>>> MyClass.common = 30
>>> classinstance.common
30
>>> classinstance2.common
30
# This will not update the variable on the class,
# instead it will bind a new object to the old
# variable name.
>>> classinstance.common = 10
>>> classinstance.common
10
>>> classinstance2.common
30
>>> MyClass.common = 50
# This has not changed, because "common" is
# now an instance variable.
>>> classinstance.common
10
>>> classinstance2.common
50
```

```
# This class inherits from MyClass. The example
# class above inherits from "object", which makes
# it what's called a "new-style class".
# Multiple inheritance is declared as:
# class OtherClass(MyClass1, MyClass2, MyClassN)
class OtherClass(MyClass):
    # The "self" argument is passed automatically
    # and refers to the class instance, so you can set
    # instance variables as above, but from inside the class.
    def __init__(self, arg1):
        self.myvariable = 3
        print(arg1)

>>> classinstance = OtherClass("hello")
hello
>>> classinstance.myfunction(1, 2)
3
# This class doesn't have a .test member, but
# we can add one to the instance anyway. Note
# that this will only be a member of classinstance.
>>> classinstance.test = 10
>>> classinstance.test
10
```

7. **File I/O:** Python has a wide array of libraries built in. As an example, here is how serializing (converting data structures to strings using the pickle library) with file I/O is used:

```

import pickle
mylist = ["This", "is", 4, 13327]
# Open the file C:\\binary.dat for writing. The letter r before the
# filename string is used to prevent backslash escaping.
myfile = open(r"C:\\binary.dat", "wb")
pickle.dump(mylist, myfile)
myfile.close()

myfile = open(r"C:\\text.txt", "w")
myfile.write("This is a sample string")
myfile.close()

myfile = open(r"C:\\text.txt")
>>> print(myfile.read())
'This is a sample string'
myfile.close()

# Open the file for reading.
myfile = open(r"C:\\binary.dat", "rb")
loadedlist = pickle.load(myfile)
myfile.close()
>>> print(loadedlist)
['This', 'is', 4, 13327]

```

2) Solved Lab Activites

<i>Sr.No</i>	<i>Allocated Time</i>	<i>Level of Complexity</i>	<i>CLO Mapping</i>
<i>1</i>	<i>10</i>	<i>Low</i>	<i>CLO-2</i>
<i>2</i>	<i>10</i>	<i>Low</i>	<i>CLO-2</i>
<i>3</i>	<i>10</i>	<i>Low</i>	<i>CLO-2</i>
<i>4</i>	<i>10</i>	<i>Low</i>	<i>CLO-2</i>
<i>5</i>	<i>10</i>	<i>Low</i>	<i>CLO-2</i>
<i>6</i>	<i>10</i>	<i>Low</i>	<i>CLO-2</i>

Activity 1:

Python program to illustrate data types.

Solution:

```

>>> sample = [1, ["another", "list"], ("a", "tuple")]
>>> mylist = ["List item 1", 2, 3.14]
>>> mylist[0] = "List item 1 again" # We're changing the item.
>>> mylist[-1] = 3.21 # Here, we refer to the last item.
>>> mydict = {"Key 1": "Value 1", 2: 3, "pi": 3.14}
>>> mydict["pi"] = 3.15 # This is how you change dictionary values.

```

```
>>> mytuple = (1, 2, 3)
>>> myfunction = len
>>> print(myfunction(mylist))
```

Output

Value of myfunction:3

Activity 2:

Python program to illustrate indexing and slicing.

```
>>> mylist = ["List item 1", 2, 3.14]
>>> print(mylist[:])
['List item 1', 2, 3.1400000000000001]
>>> print(mylist[0:2])
['List item 1', 2]
>>> print(mylist[-3:-1])
['List item 1', 2]
>>> print(mylist[1:])
[2, 3.14]
# Adding a third parameter, "step" will have Python step in
# N item increments, rather than 1.
# E.g., this will return the first item, then go to the third and
# return that (so, items 0 and 2 in 0-indexing).
>>> print(mylist[::2])
['List item 1', 3.14]
```

Output

1. Slicing the Entire List:

- `mylist[:]` returns all elements of `mylist`.
- **Output:** `['List item 1', 2, 3.14]`

2. Slicing from Index 0 to Index 2 (Exclusive):

- `mylist[0:2]` returns elements from the beginning up to, but not including, index 2.
- **Output:** `['List item 1', 2]`

3. Slicing with Negative Indices:

- `mylist[-3:-1]` returns elements from the third-to-last element up to, but not including, the last element.
- **Output:** `['List item 1', 2]`

4. Slicing from Index 1 to the End:

- `mylist[1:]` returns elements from index 1 to the end of the list.
- **Output:** `[2, 3.14]`

5. Slicing with Step Parameter:

- `mylist[::2]` returns every second element of the list, starting from the first element.
- **Output:** `['List item 1', 3.14]`

Activity 3:

python program to illustrate flow control statement.

```
>>> print(range(10))
range(0, 10)
>>> rangelist = list(range(10))
>>> print(rangelist)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

for number in range(10):
    # Check if number is one of
    # the numbers in the tuple.
    if number in (3, 4, 7, 9):
        # "Break" terminates a for without
        # executing the "else" clause.
        break
    else:
        # "Continue" starts the next iteration
        # of the loop. It's rather useless here,
        # as it's the last statement of the loop.
        continue
else:
    # The "else" clause is optional and is
    # executed only if the loop didn't "break".
    pass # Do nothing

if rangelist[1] == 2:
    print("The second item (lists are 0-based) is 2")
elif rangelist[1] == 3:
    print("The second item (lists are 0-based) is 3")
else:
    print("Dunno")

while rangelist[1] == 1:
    print("We are trapped in an infinite loop!")
```

Output

1. `range(0, 10)`
2. `[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]`
3. `Dunno`

-
4. We are trapped in an infinite loop!
 5. We are trapped in an infinite loop!
 6. We are trapped in an infinite loop! ...

Activity 4:

python program to illustrate function.

```
# Same as def funcvar(x): return x + 1
funcvar = lambda x: x + 1
>>> print(funcvar(1))
2

# an_int and a_string are optional, they have default values
# if one is not passed (2 and "A default string", respectively).
def passing_example(a_list, an_int=2, a_string="A default string"):
    a_list.append("A new item")
    an_int = 4
    return a_list, an_int, a_string

>>> my_list = [1, 2, 3]
>>> my_int = 10
>>> print(passing_example(my_list, my_int))
([1, 2, 3, 'A new item'], 4, "A default string")
>>> my_list
[1, 2, 3, 'A new item']
>>> my_int
10
```

Output

Shown above

Activity 5:

python program to illustrate class.

```
class MyClass(object):
    common = 10
    def __init__(self):
        self.myvariable = 3
    def myfunction(self, arg1, arg2):
        return self.myvariable

# This is the class instantiation

>>> classinstance = MyClass()
```

```

>>> classinstance.myfunction(1, 2)
3
# This variable is shared by all instances.
>>> classinstance2 = MyClass()
>>> classinstance.common
10
>>> classinstance2.common
10
# Note how we use the class name
# instead of the instance.
>>> MyClass.common = 30
>>> classinstance.common
30
>>> classinstance2.common
30
# This will not update the variable on the class,
# instead it will bind a new object to the old
# variable name.
>>> classinstance.common = 10
>>> classinstance.common
10
>>> classinstance2.common
30
>>> MyClass.common = 50
# This has not changed, because "common" is
# now an instance variable.
>>> classinstance.common
10
>>> classinstance2.common
50

# This class inherits from MyClass. The example
# class above inherits from "object", which makes
# it what's called a "new-style class".
# Multiple inheritance is declared as:
# class OtherClass(MyClass1, MyClass2, MyClassN)
class OtherClass(MyClass):
    # The "self" argument is passed automatically
    # and refers to the class instance, so you can set
    # instance variables as above, but from inside the class.
    def __init__(self, arg1):
        self.myvariable = 3
        print(arg1)

>>> classinstance = OtherClass("hello")
hello
>>> classinstance.myfunction(1, 2)

```

```
3
# This class doesn't have a .test member, but
# we can add one to the instance anyway. Note
# that this will only be a member of classinstance.
>>> classinstance.test = 10
>>> classinstance.test
10
```

Output

Shown above

Activity 6:

python program to illustrate IO handling.

```
import pickle
mylist = ["This", "is", 4, 13327]
# Open the file C:\\binary.dat for writing. The letter r before the
# filename string is used to prevent backslash escaping.
myfile = open(r"C:\\binary.dat", "wb")
pickle.dump(mylist, myfile)
myfile.close()

myfile = open(r"C:\\text.txt", "w")
myfile.write("This is a sample string")
myfile.close()

myfile = open(r"C:\\text.txt")
>>> print(myfile.read())
'This is a sample string'
myfile.close()

# Open the file for reading.
myfile = open(r"C:\\binary.dat", "rb")
loadedlist = pickle.load(myfile)
myfile.close()
>>> print(loadedlist)
['This', 'is', 4, 13327]
```

Output

Shown above

3) Graded Lab Tasks

The purpose of this lab task is to help students understand and manipulate different data structures in Python, specifically lists, dictionaries, and tuples. Students will perform various operations on these data structures, including creation, modification, and data access.

Lab Task 1

Instructions:

1. List Operations:

- Create a list with the following elements: ["red", "green", "blue", "yellow", "purple"].
- Add "orange" to the end of the list.
- Insert "pink" at the second position.
- Remove "blue" from the list.
- Reverse the list.
- Sort the list in alphabetical order.
- Slice the list to get the last three elements.
- Print the length of the list.

2. Dictionary Operations:

- Create a dictionary with the following key-value pairs: {"name": "John", "age": 25, "city": "New York"}.
- Add a new key-value pair for "email": "john.doe@example.com".
- Update the "age" to 26.
- Remove the "city" key from the dictionary.
- Check if the key "name" exists in the dictionary.
- Print all the keys and values of the dictionary.

3. Tuple Operations:

- Create a tuple with the following elements: (10, 20, 30, 40, 50).
- Access and print the third element of the tuple.
- Try to change the second element of the tuple and observe the result (Note: Tuples are immutable).
- Create a new tuple by concatenating the existing tuple with (60, 70, 80).
- Unpack the tuple into individual variables and print them.

4. Combining Data Structures:

- Create a dictionary where the keys are strings and the values are lists. For example: {"fruits": ["apple", "banana", "cherry"], "vegetables": ["carrot", "broccoli", "spinach"]}.
- Add a new fruit "orange" to the list of fruits.
- Remove "spinach" from the list of vegetables.
- Print the updated dictionary.
- Create a list of tuples, where each tuple contains a fruit and its color, e.g., [("apple", "red"), ("banana", "yellow"), ("cherry", "red")].
- Access and print the color of "banana" from the list of tuples.

Submission:

1. Create a Python script file (e.g., `lab_data_structures.py`) containing all the operations mentioned above.
2. Add comments in your script explaining each step.
3. Test your script to ensure all tasks are performed correctly.
4. Submit the Python script file along with a brief report (1-2 pages) explaining your observations and any challenges you faced during the lab.

Lab Task 2

The purpose of this lab task is to help students understand how to define and use functions and classes in Python. Students will create functions for specific tasks and design classes that encapsulate data and behavior.

Instructions:

1. Function Definition and Usage:

- Write a function `greet` that takes a name as an argument and prints a greeting message.
- Write a function `fibonacci` that takes an integer `n` and returns the `n`th Fibonacci number.
- Write a function `is_prime` that takes an integer and returns `True` if the number is prime, and `False` otherwise.
- Write a function `factorial` that takes an integer `n` and returns the factorial of `n`.
- Write a function `reverse_string` that takes a string and returns the reversed string.

2. Class Definition and Usage:

- Define a class `Person` with the following attributes: `name`, `age`, and `email`.
- Add a method `introduce` to the `Person` class that prints an introduction message using the person's attributes.
- Define a class `Student` that inherits from `Person` and adds an attribute `student_id`.
- Add a method `study` to the `Student` class that prints a message indicating the student is studying.
- Define a class `Course` with attributes `course_name`, `students` (a list of `Student` objects), and a method `add_student` to add a student to the course.
- Add a method `list_students` to the `Course` class that prints the names of all students enrolled in the course.

3. Combining Functions and Classes:

- Create an instance of the `Course` class with the course name "Introduction to Python".
- Create three instances of the `Student` class with different names, ages, emails, and student IDs.
- Add the students to the course using the `add_student` method.
- List all students in the course using the `list_students` method.
- Use the `introduce` method to print introduction messages for each student.
- Demonstrate the `study` method for one of the students.

Submission:

1. Create a Python script file (e.g., `lab_functions_classes.py`) containing all the operations mentioned above.
2. Add comments in your script explaining each step.
3. Test your script to ensure all tasks are performed correctly.
4. Submit the Python script file along with a brief report (1-2 pages) explaining your observations and any challenges you faced during the lab.

Lab Task 3

The purpose of this lab task is to help students understand how to perform input/output (I/O) operations using the pickle module in Python. Students will learn how to serialize (pickle) and deserialize (unpickle) Python objects and store them in files.

Instructions:

1. **Understanding Pickling:**
 - Write a brief explanation (in comments or a separate markdown file) of what pickling is and why it is used in Python.
2. **Basic Pickling and Unpickling:**
 - Create a list of dictionaries where each dictionary represents a student with keys `name`, `age`, and `grade`. For example:

```
python
Copy code
students = [
    {"name": "Alice", "age": 20, "grade": "A"},
    {"name": "Bob", "age": 22, "grade": "B"},
    {"name": "Charlie", "age": 19, "grade": "C"}
]
```
 - Use the `pickle` module to serialize the list of students and save it to a file called `students.pkl`.
 - Write a function `load_students` that deserializes the data from `students.pkl` and returns the list of students.
 - Print the loaded students to verify the data.
3. **Working with Custom Classes:**
 - Define a class `Student` with attributes `name`, `age`, and `grade`.
 - Modify the previous list of dictionaries to a list of `Student` objects.
 - Serialize the list of `Student` objects and save it to a file called `students_class.pkl`.
 - Write a function `load_student_objects` that deserializes the data from `students_class.pkl` and returns the list of `Student` objects.
 - Print the loaded student objects to verify the data.
4. **Advanced Operations:**
 - Write a function `add_student` that takes a file name, a `Student` object, and adds the student to the list in the file.

-
- Write a function `remove_student` that takes a file name and a student name, and removes the student with that name from the file.
 - Demonstrate the usage of `add_student` and `remove_student` functions by adding a new student and removing an existing student from `students_class.pkl`.

Submission:

1. Create a Python script file (e.g., `lab_pickle_io.py`) containing all the operations mentioned above.
2. Add comments in your script explaining each step.
3. Test your script to ensure all tasks are performed correctly.
4. Submit the Python script file along with a brief report (1-2 pages) explaining your observations and any challenges you faced during the lab.

Lab 02

Objective:

The objective of this lab is to learn about numpy, a python library, commonly used for implementing neural networks.

Activity Outcomes:

The activities provide hands - on practice with the following topics

- Implement vectors and matrixes.
- Perform vectors and matrix operations such as dot product, addition, subtraction, reshaping, etc
- Perform indexing and slicing.
- Implement random matrixes
- Implement algebraic operations such as transpose

Instructor Note:

As pre-lab activity, read and implement the tutorial:

<https://numpy.org/doc/stable/user/quickstart.html> .

4) Useful Concepts

1. **NumPy (Numerical Python):** It is a powerful Python library used extensively in numerical computing and data analysis. It provides support for large, multi-dimensional arrays and matrices, along with a collection of high-level mathematical functions to operate on these arrays efficiently.

Important Characteristics of NumPy:

- i. **Multi-dimensional Arrays:** NumPy's primary data structure is the ndarray, a multi-dimensional array object that can hold elements of the same type.
- ii. **Efficient Operations:** NumPy operations are implemented in C and Fortran, making them fast and efficient compared to Python's built-in data structures.
- iii. **Mathematical Functions:** NumPy provides a rich set of mathematical functions that operate element-wise on arrays, facilitating complex mathematical computations.
- iv. **Broadcasting:** NumPy supports broadcasting, which allows arithmetic operations on arrays of different shapes without explicitly converting them.
- v. **Integration with Other Libraries:** NumPy integrates well with other Python libraries for data analysis, such as pandas, matplotlib, and scikit-learn.

To use NumPy in your Python programs, you need to import it first. Here's how you can import NumPy:

```
import numpy as np
```

2. **Vector:** A vector, in the context of NumPy in Python, is a one-dimensional array that can store a sequence of numbers. It is a fundamental data structure used in various mathematical and scientific computations. In NumPy, you can create a vector using the `numpy.array` function:

```
#Load Library
import numpy as np

#Create a vector as a Row
vector_row = np.array([1,2,3])

#Create vector as a Column
vector_column = np.array([[1],[2],[3]])
```

-
3. **Matrix:** A matrix refers to a specific type of two-dimensional array. It is a fundamental data structure used for various mathematical operations, especially for implementing artificial neural networks. A matrix in NumPy is represented by the *numpy.ndarray* object with two dimensions.

It can be visualized as a rectangular grid of numbers arranged in rows and columns.

```
import numpy as np

# Creating a 2x3 matrix (2 rows, 3 columns)
matrix = np.array([[1, 2, 3], [4, 5, 6]])
print(matrix)
```

Element-wise Operations: Matrices support element-wise arithmetic operations (addition, subtraction, multiplication, division) similar to vectors but applied to corresponding elements.

```
matrix1 = np.array([[1, 2], [3, 4]])
matrix2 = np.array([[5, 6], [7, 8]])

# Element-wise addition
result = matrix1 + matrix2
print(result)
```

Matrix Multiplication: For matrix multiplication (dot product), you can use the `np.dot` function or the `@` operator in Python 3.5+:

```
result = np.dot(matrix1, matrix2)
# OR
result = matrix1 @ matrix2
print(result)
```

4. **Creating a Sparse Matrix:** Given data with very few non zero values you want to efficiently represent it. A frequent situation in Machine Learning is having a huge amount of Data; however most of the elements in the data are Zeros. For example, lets imagine a Matrix where the columns are all the products on Amazon and the rows signify whether a given user has bought that item before or not. Now as you might have guessed there would be many products which haven't been bought even a single time till now and thus a vast majority of elements would be Zero.

```
#Load Library
import numpy as np

#Create a Matrix
matrix = np.array([[0,0],[0,1],[3,0]])
print(matrix)

#Create Compressed Sparse Row(CSR) matrix
matrix_sparse = sparse.csr_matrix(matrix)
print(matrix_sparse)
```

5. **Indexing** in NumPy is the process of accessing and manipulating elements within arrays. It allows you to retrieve or modify specific elements, rows, columns, or sub-arrays in a NumPy array. Here are the key concepts:

- **Basic Indexing:**

1-D Arrays: You can access elements using their index positions, starting from 0:

```
import numpy as np

arr = np.array([10, 20, 30, 40])
print(arr[0]) # prints 10
print(arr[2]) # prints 30
```

2-D Arrays: Access elements using a pair of indices [row, column].

```
matrix = np.array([[1, 2, 3], [4, 5, 6]])
print(matrix[0, 1]) # prints 2 (first row, second column)
print(matrix[1, 2]) # prints 6 (second row, third column)
```

- **Slicing:**

Allows you to access a range of elements.

1-D Arrays: arr[start:stop:step]

```
arr = np.array([10, 20, 30, 40, 50])
print(arr[1:4]) # prints [20, 30, 40]
print(arr[:3]) # prints [10, 20, 30]
print(arr[::2]) # prints [10, 30, 50]
```

2-D Arrays: matrix[start_row:end_row, start_col:end_col]

```
matrix = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(matrix[0:2, 1:3]) # prints [[2, 3], [5, 6]]
```

- **Boolean Indexing:**

Use conditions to select elements.

```
arr = np.array([10, 20, 30, 40])
print(arr[arr > 20]) # prints [30, 40]
```

- **Fancy Indexing:**

```
arr = np.array([10, 20, 30, 40])
indices = [0, 2]
print(arr[indices]) # prints [10, 30]
```

6. Describing a Matrix

When you want to know about the shape size and dimensions of a Matrix.

```
import numpy as np

#Create a Matrix
matrix = np.array([[1,2,3],[4,5,6],[7,8,9]])

#View the Number of Rows and Columns
print(matrix.shape)

#View the number of elements (rows*columns)
print(matrix.size)

#View the number of Dimensions(2 in this case)
print(matrix.ndim)
```

7. **Applying operations to elements:** You want to apply some function to multiple elements in an array. Numpy's `vectorize` class converts a function into a function that can apply to multiple elements in an array or slice of an array.

```
#Load Library
import numpy as np

#Create a Matrix
matrix = np.array([[1,2,3],[4,5,6],[7,8,9]])
print(matrix)

#Create a function that adds 100 to something
add_100 =lambda i: i+100

#Convert it into a vectorized function
vectorized_add_100= np.vectorize(add_100)

#Apply function to all elements in matrix
print(vectorized_add_100(matrix))
```

8. **Finding the max and min values:** We use Numpy's max and min functions:

```
#Load Library
import numpy as np

#Create a Matrix
matrix = np.array([[1,2,3],[4,5,6],[7,8,9]])
print(matrix)

#Return the max element
print(np.max(matrix))

#Return the min element
print(np.min(matrix))

#To find the max element in each column
print(np.max(matrix,axis=0))

#To find the max element in each row
print(np.max(matrix,axis=1))
```

9. **Calculating Average, Variance and Standard deviation:** When you want to calculate some descriptive statistics about an array.

```
#Load Library
import numpy as np

#Create a Matrix
matrix = np.array([[1,2,3],[4,5,6],[7,8,9]])
print(matrix)

#Mean
print(np.mean(matrix))

#Standard Dev.
print(np.std(matrix))

#Variance
print(np.var(matrix))
```

10. Reshaping Arrays: When you want to reshape an array(changing the number of rows and columns) without changing the elements.

```
#Load Library
import numpy as np

#Create a Matrix
matrix = np.array([[1,2,3],[4,5,6],[7,8,9]])
print(matrix)

#Reshape
print(matrix.reshape(9,1))

#Here -1 says as many columns as needed and 1 row
print(matrix.reshape(1,-1))

#If we provide only 1 value Reshape would return a 1-d array of that
length
print(matrix.reshape(9))

#We can also use the Flatten method to convert a matrix to 1-d array
print(matrix.flatten())
```

11. Transposing a vector or a Matrix: By transposing you interchange the rows and columns of a Matrix

```
#Load Library
import numpy as np

#Create a Matrix
matrix = np.array([[1,2,3],[4,5,6],[7,8,9]])
print(matrix)

#Transpose the matrix
print(matrix.T)
```

12. Finding the Determinant and Rank of a Matrix: The rank of a Matrix is the number of dimensions of the vector space spanned by its rows or columns.

```
#Load Library
import numpy as np

#Create a Matrix
matrix = np.array([[1,2,3],[4,5,6],[7,8,9]])
print(matrix)

#Calculate the Determinant
print(np.linalg.det(matrix))

#Calculate the Rank
print(np.linalg.matrix_rank(matrix))
```

13. Getting the Diagonal of a Matrix: When you need to extract only the diagonal elements of a matrix

```
#Load Library
import numpy as np

#Create a Matrix
matrix = np.array([[1,2,3],[4,5,6],[7,8,9]])
print(matrix)

#Print the Principal diagonal
print(matrix.diagonal())

#Print the diagonal one above the Principal diagonal
print(matrix.diagonal(offset=1))

#Print the diagonal one below Principal diagonal
print(matrix.diagonal(offset=-1))
```

14. Calculating Dot Products

```

#Load Library
import numpy as np

#Create vector-1
vector_1 = np.array([ 1,2,3 ])

#Create vector-2
vector_1 = np.array([ 4,5,6 ])

#Calculate Dot Product
print(np.dot(vector_1,vector_2))

#Alternatively you can use @ to calculate dot products
print(vector_1 @ vector_2)

```

15. **Generating Random values:** Numpy offers a wide variety of means to generate Random Numbers. Moreover, It can sometimes be useful to return the same random numbers to get predictable, repeatable results. We can do so by setting the ‘Seed’ (An Integer) of the pseudorandom generator. Random processes with the same seed would always produce the same result.

```

#Load Library
import numpy as np

#Set seed
np.random.seed(1)

#Generate 3 random integers b/w 1 and 10
print(np.random.randint(0,11,3))

#Draw 3 numbers from a normal distribution with mean 1.0 and std 2.0
print(np.random.normal(1.0,2.0,3))

```

16. Solved Lab Activites

<i>Sr.No</i>	<i>Allocated Time</i>	<i>Level of Complexity</i>	<i>CLO Mapping</i>
1	10	Low	CLO-2
2	10	Low	CLO-2
3	10	Low	CLO-2
4	10	Low	CLO-2
5	10	Low	CLO-2
6	10	Low	CLO-2
7	10	Low	CLO-2
8	10	Low	CLO-2
9	10	Low	CLO-2

Activity 1:

Numpy to illustrate 2-D Matrix.

Solution:

```
#Load Library
import numpy as np

#Create a Matrix
matrix = np.array([[1,2,3],[4,5,6]])
print(matrix)
```

Activity 2:

Numpy to illustrate indexing and slicing.

```
#Load Library
import numpy as np

#Create a vector as a Row
vector_row = np.array([ 1,2,3,4,5,6 ])

#Create a Matrix
matrix = np.array([[1,2,3],[4,5,6],[7,8,9]])
print(matrix)

#Select 3rd element of Vector
print(vector_row[2])

#Select 2nd row 2nd column
print(matrix[1,1])#Select all elements of a vector
print(vector_row[:])#Select everything up to and including the
3rd element
print(vector_row[:3])#Select the everything after the 3rd element
print(vector_row[3:])#Select the last element
print(vector_row[-1])#Select the first 2 rows and all the columns of
the matrix
print(matrix[:2,:])#Select all rows and the 2nd column of the
matrix
print(matrix[:,1:2])
```

Activity 3:

Describing the matrix.

```
import numpy as np
```

```
#Create a Matrix
matrix = np.array([[1,2,3],[4,5,6],[7,8,9]])#View the Number of
Rows and Columns
print(matrix.shape)#View the number of elements (rows*columns)
print(matrix.size)#View the number of Dimensions(2 in this case)
print(matrix.ndim)
```

Activity 4:

Finding mix and max values.

```
#Load Library
import numpy as np

#Create a Matrix
matrix = np.array([[1,2,3],[4,5,6],[7,8,9]])
print(matrix)#Return the max element
print(np.max(matrix))#Return the min element
print(np.min(matrix))#To find the max element in each column
print(np.max(matrix,axis=0))#To find the max element in each row
print(np.max(matrix,axis=1))
```

Output

Shown above

Activity 5:

Reshaping array.

```
#Load Library
import numpy as np

#Create a Matrix
matrix = np.array([[1,2,3],[4,5,6],[7,8,9]])
print(matrix)#Reshape
print(matrix.reshape(9,1))#Here -1 says as many columns as needed
and 1 row
print(matrix.reshape(1,-1))#If we provide only 1 value Reshape
would return a 1-d array of that lengthprint(marix.reshape(9))#We
can also use the Flatten method to convert a matrix to 1-d
arrayprint(matrix.flatten())
```

Output

Shown above

Activity 6:

python program to transpose an array.

```
#Load Library
import numpy as np

#Create a Matrix
matrix = np.array([[1,2,3],[4,5,6],[7,8,9]])
print(matrix)#Transpose the matrix
print(matrix.T)
```

Output

Shown above

Activity 7:

Finding dot product.

```
#Load Library
import numpy as np

#Create vector-1
vector_1 = np.array([ 1,2,3 ])#Create vector-2
vector_2 = np.array([ 4,5,6 ])#Calculate Dot Product
print(np.dot(vector_1,vector_2))#Alternatively you can use @ to
calculate dot products
print(vector_1 @ vector_2)
```

Output

Shown above

Activity 8:

Finding dot product.

```
#Load Library
import numpy as np

#Create vector-1
vector_1 = np.array([ 1,2,3 ])#Create vector-2
vector_2 = np.array([ 4,5,6 ])#Calculate Dot Product
print(np.dot(vector_1,vector_2))#Alternatively you can use @ to
calculate dot products
print(vector_1 @ vector_2)
```

Output

Shown above

Activity 9:

Generating random values.

```
#Load Library
import numpy as np#Set seed
np.random.seed(1)#Generate 3 random integers b/w 1 and 10
print(np.random.randint(0,11,3))#Draw 3 numbers from a normal
distribution with mean 1.0 and std 2.0
print(np.random.normal(1.0,2.0,3))
```

Output

Shown above

17. Graded Lab Tasks

The purpose of this lab task is to practice creating, accessing, and slicing matrices using NumPy in Python.

Lab Task 1

Instructions:

- Matrix Declaration:**
 - Create a 3x3 matrix of integers.
 - Create a 4x4 matrix of floats.
- Matrix Indexing:**
 - Access the element at the 2nd row and 3rd column of the integer matrix.
 - Change the element at the 1st row and 4th column of the float matrix to 20.5.
- Matrix Slicing:**
 - Extract the first two rows and columns of the integer matrix.
 - Extract the last row of the float matrix.

Submission:

- Save your code in a Python script named `matrix_lab.py`.
- Submit your script file by the due date.

Lab Task 2

The purpose of this lab task is to help students to practice creating matrices and performing various matrix operations using NumPy in Python.

Instructions:

- Matrix Declaration:**

Create the following matrices:

- A 3x3 integer matrix named `matrix_int`.
- A 3x3 float matrix named `matrix_float`.

2. Basic Matrix Operations:

- Perform element-wise addition of `matrix_int` and `matrix_float`.
- Perform element-wise multiplication of `matrix_int` and `matrix_float`.
- Find the transpose of `matrix_int`.

3. Advanced Matrix Operations:

- Perform matrix multiplication of `matrix_int` and `matrix_float`.
- Calculate the determinant of `matrix_int`.
- Find the inverse of `matrix_float`.

Submission:

- Save your code in a Python script named `matrix_operations_lab.py`.
- Submit your script file by the due date.

Lab Task 3

For this project, you will use a simple dataset of student grades. The dataset is provided below as a list of lists, where each sublist represents a student and contains their ID, math grade, science grade, and English grade.

```
data = [ [1, 85, 78, 92], [2, 76, 85, 88], [3, 90, 92, 95], [4, 68, 72, 70], [5, 88, 89, 94] ]
```

Instructions:

- **Setup:**
 - Import the necessary libraries: `numpy` and `matplotlib`.
- **Load and Display the Dataset:**
 - Convert the provided dataset into a NumPy array.
 - Display the dataset.
- **Extract and Analyze Specific Data Columns:**
 - Extract the math, science, and English grades as separate arrays.
 - Calculate and display the average grade for each subject.
- **Perform Basic Statistical Analysis:**

-
- Calculate the overall average grade for each student.
 - Find the student with the highest and lowest average grade.
 - **Manipulate and Transform the Data:**
 - Add 5 bonus points to each student's math grade.
 - Calculate the new average grade for each student after adding the bonus points.

Submission:

- Save your code in a Python script named `data_analysis_lab.py`.
- Submit your script file by the due date.

Lab 03

Objective:

The objective of this lab is to learn how to implement perceptron – a basic building block of neural networks – in python.

Activity Outcomes:

The activities provide hands - on practice with the following topics

- Implement a perceptron model.
- Train perceptron using a dataset.

Instructor Note:

As pre-lab activity, read and implement the following tutorials:

1. <https://towardsdatascience.com/perceptron-and-its-implementation-in-python-f87d6c7aa428>
2. <https://machinelearningmastery.com/implement-perceptron-algorithm-scratch-python/>
3. <https://www.kaggle.com/code/kuzagaya/implementing-a-perceptron-in-python>

5) Useful Concepts

Perceptron: It is a system that learns using labeled examples (i.e., supervised learning) of feature vectors (or raw pixel intensities), mapping these inputs to their corresponding output class labels. In its simplest form, a Perceptron contains N input nodes, one for each entry in the input row of the design matrix, followed by only one layer in the network with just a single node in that layer (Figure 1).

There exist connections and their corresponding weights $w_1, w_2, \dots, w_i$ from the input x_i 's to the single output node in the network. This node takes the weighted sum of inputs and applies a step function to determine the output class label. The Perceptron outputs either a 0 or a 1-0 for class #1 and 1 for class #2; thus, in its original form, the Perceptron is simply a binary, two-class classifier.

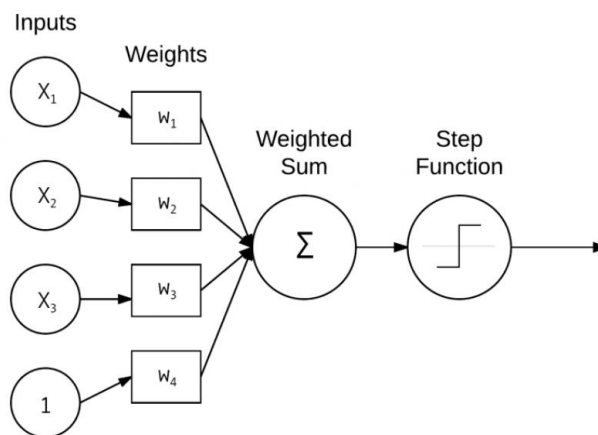


Figure 1: Architecture of perceptron.

Perceptron Training Procedure and the Delta Rule: Training a Perceptron is a fairly straightforward operation. Our goal is to obtain a set of weights $\mathbf{w}$ that accurately classifies each instance in our training set. In order to train our Perceptron, we iteratively feed the network with our training data *multiple times*. Each time the network has seen the *full set* of training data, we say an *epoch* has passed. It normally takes many epochs until a weight vector $\mathbf{w}$ can be learned to linearly separate our two classes of data. The pseudocode for the Perceptron training algorithm can be found below:

1. Initialize our weight vector $\mathbf{w}$ with small random values
2. Until Perceptron converges:
 - (a) Loop over each feature vector $\mathbf{x}_j$ and true class label d_j in our training set D
 - (b) Take $\mathbf{x}$ and pass it through the network, calculating the output value: $y_j = f(\mathbf{w}(\mathbf{t}) \cdot \mathbf{x}_j)$
 - (c) Update the weights $\mathbf{w}$: $w_i(t+1) = w_i(t) + \alpha(d_j - y_j)x_{j,i}$ for all features $0 \leq i \leq n$

Figure 2: Perceptron algorithm training procedure.

The actual “learning” takes place in Steps 2b and 2c. First, we pass the feature vector x_j through the network, take the dot product with the weights $\mathbf{w}$ and obtain the output y_j . This value is then passed through the step function which will return 1 if $x > 0$ and 0 otherwise.

Now we need to update our weight vector $\mathbf{w}$ to step in the direction that is “closer” to the correct classification. This update of the weight vector is handled by the *delta rule* in Step 2c.

The expression $(dj - yj)$ determines if the output classification is correct or not. If the classification is *correct*, then this difference will be zero. Otherwise, the difference will be either positive or negative, giving us the direction in which our weights will be updated (ultimately bringing us closer to the correct classification). We then multiply $(dj - yj)$ by x_j , moving us closer to the correct classification.

The value α is our *learning rate* and controls how large (or small) of a step we take. It’s *critical* that this value is set correctly. A larger value of α will cause us to take a step in the right direction; however, this step could be *too large*, and we could easily overstep a local/global optimum.

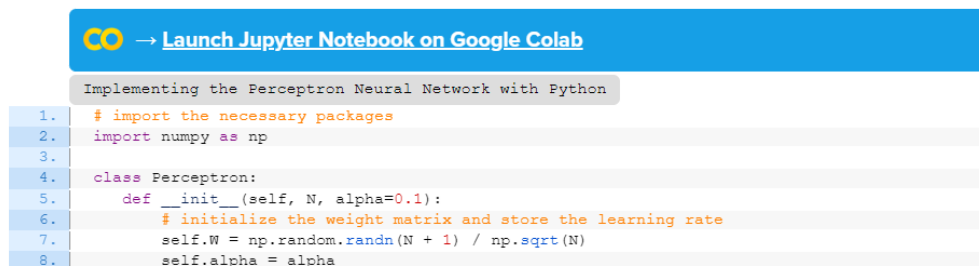
Conversely, a small value of α allows us to take tiny baby steps in the right direction, ensuring we don’t overstep a local/global minimum; however, these tiny baby steps may take an intractable amount of time for our learning to converge.

Finally, we add in the previous weight vector at time t , $\mathbf{w}_j(t)$ which completes the process of “stepping” towards the correct classification.

Understanding the Implementation of Perceptron in Python:

Now that we have studied the Perceptron algorithm, let’s implement the actual algorithm in Python.

Step 1: Create a file named `perceptron.py` in your jupyter notebook. After you’ve created the file, open it, and insert the following code:



```
CO → Launch Jupyter Notebook on Google Colab
Implementing the Perceptron Neural Network with Python
1. # import the necessary packages
2. import numpy as np
3.
4. class Perceptron:
5.     def __init__(self, N, alpha=0.1):
6.         # initialize the weight matrix and store the learning rate
7.         self.W = np.random.randn(N + 1) / np.sqrt(N)
8.         self.alpha = alpha
```

Line 5 defines the constructor to our Perceptron class, which accepts a single required parameter followed by a second optional one:

- N: The number of columns in our input feature vectors. In the context of our bitwise datasets, we'll set

N equal to two since there are two inputs.

- alpha: Our learning rate for the Perceptron algorithm. We'll set this value to 0.1 by default. Common choices of learning rates are normally in the range $\alpha = 0.1, 0.01, 0.001$.

Line 7 files our weight matrix W with random values sampled from a “normal” (Gaussian) distribution with zero mean and unit variance. The weight matrix will have N + 1 entries, one for each of the N inputs in the feature vector, plus one for the bias. We divide W by the square-root of the number of inputs, a common technique used to scale our weight matrix, leading to faster convergence. We will cover weight initialization techniques later in this chapter.

Step 2: Next, let's define the step function.

 → [Launch Jupyter Notebook on Google Colab](#)

Implementing the Perceptron Neural Network with Python

```

10.     def step(self, x):
11.         # apply the step function
12.         return 1 if x > 0 else 0

```

This function mimics the behavior of the step equation — if x is positive we return 1, otherwise, we return 0.

Step 3: To actually train the Perceptron we'll define a function named `fit`. If you have any previous experience with machine learning, Python, and the scikit-learn library then you'll know that it's common to name your training procedure function `fit`, as in “fit a model to the data”:

 → [Launch Jupyter Notebook on Google Colab](#)

Implementing the Perceptron Neural Network with Python

```

14.     def fit(self, X, y, epochs=10):
15.         # insert a column of 1's as the last entry in the feature
16.         # matrix -- this little trick allows us to treat the bias
17.         # as a trainable parameter within the weight matrix
18.         X = np.c_[X, np.ones((X.shape[0]))]

```

The `fit` method requires two parameters followed by a single optional one: The X value is our actual training data. The y variable is our target output class labels (i.e., what our network should be predicting). Finally, we supply `epochs`, the number of epochs our Perceptron will train for.

Line 18 applies the bias trick by inserting a column of ones into the training data, which allows us to treat the bias as a trainable parameter directly inside the weight matrix.

Step 4: Let's review the actual training procedure:

 → [Launch Jupyter Notebook on Google Colab](#)

Implementing the Perceptron Neural Network with Python

```
20.         # loop over the desired number of epochs
21.     for epoch in np.arange(0, epochs):
22.         # loop over each individual data point
23.         for (x, target) in zip(X, y):
24.             # take the dot product between the input features
25.             # and the weight matrix, then pass this value
26.             # through the step function to obtain the prediction
27.             p = self.step(np.dot(x, self.W))
28.
29.             # only perform a weight update if our prediction
30.             # does not match the target
31.             if p != target:
32.                 # determine the error
33.                 error = p - target
34.
35.                 # update the weight matrix
36.                 self.W += -self.alpha * error * x
```

On Line 21, we start looping over the desired number of epochs. For each epoch, we also loop over each individual data point x and output target class label (Line 23). Line 27 takes the dot product between the input features x and the weight matrix W , then passes the output through the step function to obtain the prediction by the Perceptron. Applying the same training procedure detailed in Figure 3, we only perform a weight update if our prediction does not match the target (Line 31). If this is the case, we determine the error (Line 33) by computing the sign (either positive or negative) via the difference operation. Updating the weight matrix is handled on Line 36 where we take a step towards the correct classification, scaling this step by our learning rate α . Over a series of epochs, our Perceptron is able to learn patterns in the underlying data and shift the values of the weight matrix such that we correctly classify our input samples x .

Step 5: Lets understand the implementation of predict function.

 → [Launch Jupyter Notebook on Google Colab](#)

Implementing the Perceptron Neural Network with Python

```
38.     def predict(self, X, addBias=True):
39.         # ensure our input is a matrix
40.         X = np.atleast_2d(X)
41.
42.         # check to see if the bias column should be added
43.         if addBias:
44.             # insert a column of 1's as the last entry in the feature
45.             # matrix (bias)
46.             X = np.c_[X, np.ones((X.shape[0]))]
47.
48.         # take the dot product between the input features and the
49.         # weight matrix, then pass the value through the step
50.         # function
51.         return self.step(np.dot(X, self.W))
```

Our predict method requires a set of input data X that needs to be classified. A check on Line 43 is made to see if a bias column needs to be added. Obtaining the output predictions for X is the same as the training procedure — simply take the dot product between the input features X and our weight matrix W , and then pass the value through our step function. The output of the step function is returned to the calling function. Now that we've implemented our Perceptron class, let's try to apply it to our bitwise datasets and see how the neural network performs.

Evaluating the Perceptron Bitwise Datasets: To start, let's create a file named `perceptron_or.py` that attempts to fit a Perceptron model to the bitwise OR dataset:

CO → [Launch Jupyter Notebook on Google Colab](#)

Implementing the Perceptron Neural Network with Python

```
1. # import the necessary packages
2. from pyimagesearch.nn import Perceptron
3. import numpy as np
4.
5. # construct the OR dataset
6. X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
7. y = np.array([[0], [1], [1], [1]])
8.
9. # define our perceptron and train it
10. print("[INFO] training perceptron...")
11. p = Perceptron(X.shape[1], alpha=0.1)
12. p.fit(X, y, epochs=20)
```

Lines 2 and 3 import our required Python packages. We'll be using our Perceptron implementation. Lines 6 and 7 define the OR dataset based on Table 1. Lines 11 and 12 train our Perceptron with a learning rate of $\alpha = 0.1$ for a total of 20 epochs. We can then evaluate our Perceptron on the data to validate that it did, in fact, learn the OR function:

CO → [Launch Jupyter Notebook on Google Colab](#)

```
14. # now that our perceptron is trained we can evaluate it
15. print("[INFO] testing perceptron...")
16.
17. # now that our network is trained, loop over the data points
18. for (x, target) in zip(X, y):
19.     # make a prediction on the data point and display the result
20.     # to our console
21.     pred = p.predict(x)
22.     print("[INFO] data={}, ground-truth={}, pred={}".format(
23.         x, target[0], pred))
```

On Line 18, we loop over each of the data points in the OR dataset. For each of these data points, we pass it through the network and obtain the prediction (Line 21). Finally, Lines 22 and 23 display the input data point, the ground-truth label, as well as our predicted label to our console. To see if our Perceptron algorithm is able to learn the OR function, just execute the following command:

CO → [Launch Jupyter Notebook on Google Colab](#)

Implementing the Perceptron Neural Network with Python

```
1. $ python perceptron_or.py
2. [INFO] training perceptron...
3. [INFO] testing perceptron...
4. [INFO] data=[0 0], ground-truth=0, pred=0
5. [INFO] data=[0 1], ground-truth=1, pred=1
6. [INFO] data=[1 0], ground-truth=1, pred=1
7. [INFO] data=[1 1], ground-truth=1, pred=1
```

Again, our Perceptron was able to correctly model the function. The AND function is only true when both $x_0 = 1$ and $x_1 = 1$ – for all other combinations the bitwise AND is zero. Finally, let's take a look at the nonlinearly separable XOR function inside `perceptron_xor.py`

 → [Launch Jupyter Notebook on Google Colab](#)

Implementing the Perceptron Neural Network with Python

```

1. # import the necessary packages
2. from pyimagesearch.nn import Perceptron
3. import numpy as np
4.
5. # construct the XOR dataset
6. x = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
7. y = np.array([[0], [1], [1], [0]])
8.
9. # define our perceptron and train it
10. print("[INFO] training perceptron...")
11. p = Perceptron(X.shape[1], alpha=0.1)
12. p.fit(X, y, epochs=20)
13.
14. # now that our perceptron is trained we can evaluate it
15. print("[INFO] testing perceptron...")
16.
17. # now that our network is trained, loop over the data points
18. for (x, target) in zip(X, y):
19.     # make a prediction on the data point and display the result
20.     # to our console
21.     pred = p.predict(x)
22.     print("[INFO] data={}, ground-truth={}, pred={}".format(
23.         x, target[0], pred))

```

Again, the only lines of code that have been changed are Lines 6 and 7 where we define the XOR data. The XOR operator is true *if and only if* one (but not both) x 's are one. Executing the following command we can see that the Perceptron *cannot* learn this nonlinear relationship:

 → [Launch Jupyter Notebook on Google Colab](#)

Implementing the Perceptron Neural Network with Python

```

1. $ python perceptron_xor.py
2. [INFO] training perceptron...
3. [INFO] testing perceptron...
4. [INFO] data=[0 0], ground-truth=0, pred=1
5. [INFO] data=[0 1], ground-truth=1, pred=1
6. [INFO] data=[1 0], ground-truth=1, pred=0
7. [INFO] data=[1 1], ground-truth=0, pred=0

```

6) Solved Lab Activities

Sr.No	Allocated Time	Level of Complexity	CLO Mapping
1	25	Low	CLO-2
2	25	Low	CLO-2

Activity 1:

Load dataset: The dataset that we consider for implementing Perceptron is the Iris flower dataset. This dataset contains 4 features that describe the flower and classify them as belonging to one of the 3 classes. We strip the last 50 rows of the dataset that belongs to the class 'Iris-virginica' and use only 2 classes 'Iris-setosa' and 'Iris-versicolor' because these classes are linearly separable and the algorithm converges to a local minimum by eventually finding the optimal weights.

Solution:

```
import numpy as np
```

```
import pandas as pd

import matplotlib.pyplot as plt

def load_data():

    URL_='https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data'

    data = pd.read_csv(URL_, header = None)

    print(data)

    # make the dataset linearly separable

    data = data[:100]

    data[4] = np.where(data.iloc[:, -1]=='Iris-setosa', 0, 1)

    data = np.asmatrix(data, dtype = 'float64')

    return data

data = load_data()
```

Visualize the dataset: Visualizing the dataset with 2 of the features, we can see that that dataset can be clearly separated by drawing a straight line between them. Our goal is to write an algorithm that finds that line and classifies all of these data points correctly.

Solution:

```
plt.scatter(np.array(data[:50,0]), np.array(data[:50,2]), marker='o', label='setosa')

plt.scatter(np.array(data[50:,0]), np.array(data[50:,2]), marker='x', label='versicolor')

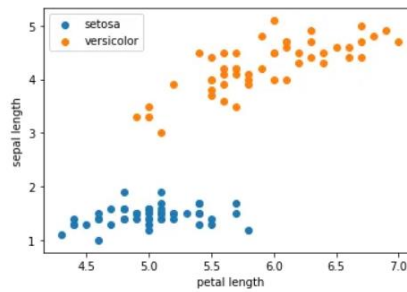
plt.xlabel('petal length')

plt.ylabel('sepal length')

plt.legend()

plt.show()
```

Output



Activity 2:

Implementation of perceptron

```
def perceptron(data, num_iter):  
    features = data[:, :-1]  
    labels = data[:, -1]  
    # set weights to zero  
    w = np.zeros(shape=(1, features.shape[1]+1))  
    misclassified_ = []  
  
    for epoch in range(num_iter):  
        misclassified = 0  
        for x, label in zip(features, labels):  
            x = np.insert(x, 0, 1)  
            y = np.dot(w, x.transpose())  
            target = 1.0 if (y > 0) else 0.0  
  
            delta = (label.item(0,0) - target)  
  
            if(delta): # misclassified  
                misclassified += 1  
                w += (delta * x)
```

```
        misclassified_.append(misclassified)

    return (w, misclassified_)

num_iter = 10

w, misclassified_ = perceptron(data, num_iter)
```

A property of the Perceptron is that if the dataset is linearly separable, then the algorithm is guaranteed to converge at some point!

```
epochs = np.arange(1, num_iter+1)

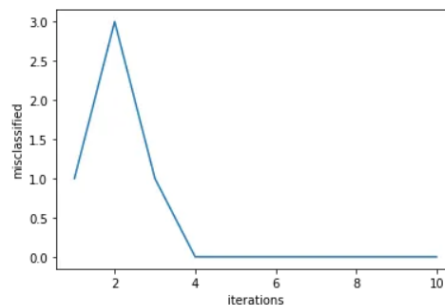
plt.plot(epochs, misclassified_)

plt.xlabel('iterations')

plt.ylabel('misclassified')

plt.show()
```

Output



7) Graded Lab Tasks

The purpose of this lab task is to help students understand, implement and train perceptron in Python.

Lab Task 1: Loading dataset: Load a dataset from Kaggle or UCI Machine Learning repository and visualize it.

Lab Task 2: Implement perceptron on the dataset, training it using training algorithm and show the learning curve and weights.

Lab 04

Objective:

The objective of this lab is to learn how to implement self organizing maps in python.

Activity Outcomes:

The activities provide hands - on practice with the following topics

- Implement SOM model in python.
- Implement training algorithm to train SOM using a dataset.

Instructor Note:

As pre-lab activity, read and implement the following tutorial:

<https://towardsdatascience.com/understanding-self-organising-map-neural-network-with-python-code-7a77f501e985>

8) Useful Concepts

Self-organizing maps: The Self-Organising Map (SOM) is an unsupervised machine learning algorithm introduced by Teuvo Kohonen in the 1980s. As the name suggests, the map organises itself without any instruction from others. It is a brain-inspired model. A different area of the cerebral cortex in our brain is responsible for specific activities. A sensory input like vision, hearing, smell, and taste is mapped to neurons of a corresponding cortex area via synapses in a self-organising way. It is also known that the neurons with similar output are in proximity. SOM is trained through a competitive neural network, a single-layer feed-forward network that resembles these brain mechanisms.

The SOM's algorithm is relatively simple, but there may be some confusion at first sight and difficulties figuring out how to apply it in practice. It may be because SOM can be understood from multiple perspectives. It is like Principal Component Analysis (PCA) for dimensionality reduction and visualisation. SOM can also be considered a type of manifold learning that handles non-linear dimensionality reduction. SOM is also used in data mining for its vector quantisation property. The train can represent high-dimensional observable data onto lower dimension latent space, typically on a 2D square grid, while preserving the topology of the original input space. But the map can also be used for projecting new data points and seeing which cluster belongs to the map, too.

Architecture and Learning Algorithm: The neural network of the Self-Organising Map has one input layer and one output layer. The second layer typically consists of a two-dimensional lattice of $m \times n$ neurons. Each neuron at the map layer is densely connected to all neurons in the input layer, possessing different weight values.

In competitive learning, neurons in the output layer compete among themselves to be activated. The winning neuron of the competition is the only one that can be fired; hence it is called the winner-takes-all neuron. In SOM, the competitive process is to search for the most similar neuron with the input pattern; the winner is called Best Matching Unit (BMU).

Similarity as the winning criterion can be measured in several ways. The most common measure employed is Euclidean distance. The neuron with the shortest distance from the input signal becomes the BMU.

In SOM, learning is not only for the winner but also for the neurons in physical proximity to it on the map. The BMU shares the privilege with its neighbours to learn together. The definition of the neighbourhood is determined by the network designer, and the optimal proximity depends on other hyperparameters. If the neighbourhood range is too small, the trained model will suffer from overfitting, and there is a risk of having some dead neurons that never have an opportunity to change.

In the adaptation phase, the BMU and its neighbours adjust their weight. The effect of the learning is to move the weight of the winning and neighbouring neurons closer to the input pattern. As an example, if the input signal is blue colour and the BMU neuron is light blue colour, the winner becomes somewhat bluer than light blue. If the neighbours are yellow, they are added a little bit of blueness to their current colour.

The Self-Organising Map learning algorithm (online learning) can be described in the following 4 steps.

1. Initialisation
Weights of neurons in the map layer are initialised.
 2. Competitive process
Select one input sample and search the best matching unit among all neurons in $n \times m$ grid using distance measures.
 3. Cooperative process
Find the proximity neurons of BMU by neighbourhood function.
 4. Adaptation process
Update the BMU and neighbours' weights by shifting the values towards the input pattern.
- If the maximum count of training iteration is reached, exit. If not, increment the iteration count by 1 and repeat the process from 2.

9) Solved Lab Activites

<i>Sr.No</i>	<i>Allocated Time</i>	<i>Level of Complexity</i>	<i>CLO Mapping</i>
1	10	Medium	CLO-3
2	10	Medium	CLO-3
3	10	Medium	CLO-3
4	10	Medium	CLO-3
5	10	Medium	CLO-3

In this lab, the SOM is trained with the Banknote Authentication data set available at the UCI ML Repository website. The data file contains 1372 rows, each with 4 features and 1 label. All 4 features are numerical values without null. The label is a binary integer value.

Firstly, we randomly split the data into training and testing data. We use all 4 features to train the SOM using an online training algorithm. Then, we evaluate the trained SOM by visualising the map using the label of the training data. Finally, we can use the test data to predict the label using the trained map.

Thus, we can demonstrate that a SOM trained in an unsupervised manner can be applied for classification using a labelled dataset.

The tutorial is available at: <https://towardsdatascience.com/understanding-self-organising-map-neural-network-with-python-code-7a77f501e985>

Activity 1: Data pre-processing

Solution:

```
import numpy as np

from sklearn import datasets

from sklearn.preprocessing import MinMaxScaler

# Load Iris dataset

data = datasets.load_iris()

X = data.data

# Normalize the dataset

scaler = MinMaxScaler()

X_scaled = scaler.fit_transform(X)
```

Activity 2: Implementing SOM

Solution:

```
class SOM:

    def __init__(self, m, n, dim, learning_rate=0.5, sigma=None):

        self.m = m
```

```
self.n = n

self.dim = dim

self.learning_rate = learning_rate

self.sigma = sigma if sigma else max(m, n) / 2.0

self.weights = np.random.rand(m, n, dim)

def _find_bmu(self, x):

    distances = np.linalg.norm(self.weights - x, axis=-1)

    bmu_index = np.unravel_index(np.argmin(distances), (self.m, self.n))

    return bmu_index

def _update_weights(self, x, bmu, iteration, max_iter):

    learning_rate = self.learning_rate * np.exp(-iteration / max_iter)

    sigma = self.sigma * np.exp(-iteration / max_iter)

    for i in range(self.m):

        for j in range(self.n):

            w = self.weights[i, j]

            distance = np.linalg.norm(np.array([i, j]) - np.array(bmu))
```

```
        if distance < sigma:

            influence = np.exp(-distance / (2 * (sigma ** 2)))

            self.weights[i, j] += influence * learning_rate * (x - w)

    def train(self, X, num_iterations):

        for iteration in range(num_iterations):

            for x in X:

                bmu = self._find_bmu(x)

                self._update_weights(x, bmu, iteration, num_iterations)
```

Activity 3: Training the SOM

Solution:

```
som = SOM(m=10, n=10, dim=4, learning_rate=0.5)

som.train(X_scaled, num_iterations=100)
```

Activity 4: Visualization

Solution:

```
import matplotlib.pyplot as plt

def plot_som(som):

    plt.figure(figsize=(10, 10))
```

```
for i in range(som.m):  
  
    for j in range(som.n):  
  
        plt.scatter(j, i, c=som.weights[i, j], marker='o', s=300, edgecolor='k')  
  
plt.title('SOM Grid')  
  
plt.show()  
  
plot_som(som)
```

Activity 5: Clustering with SOM

Solution:

```
def get_cluster_labels(som, X):  
  
    cluster_labels = []  
  
    for x in X:  
  
        bmu = som._find_bmu(x)  
  
        cluster_labels.append(bmu)  
  
    return np.array(cluster_labels)  
  
cluster_labels = get_cluster_labels(som, X_scaled)  
  
print(cluster_labels)
```

10) Graded Lab Tasks

The purpose of this lab task is to help students understand and implement self-organizing maps.

Lab Task 1: Build and Train Self-organizing Map in Python

Download a dataset from Kaggle or UCI machine learning repository and train SOMs algorithm on the dataset.

Instructions for Submission:

1. Submit a Jupyter Notebook or Python script with the complete implementation.
2. Include comments in the code to explain each step.
3. Provide a brief report (1-2 pages) summarizing the following:
 - Data preprocessing steps
 - Model architecture and rationale for chosen hyperparameters
 - Training process and evaluation metrics
 - Observations and insights from the results

Lab 05

Objective:

The objective of this lab is to learn how to implement learning vector quantization in python.

Activity Outcomes:

The activities provide hands - on practice with the following topics

- Implement LVQ model in python.
- Implement a training algorithm for LVQ in python

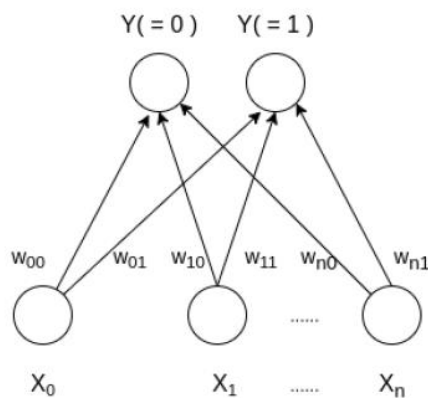
Instructor Note:

As pre-lab activity, read and implement the following tutorials:

1. <https://medium.com/@matteo.arellano/learning-vector-quantization-lvq-from-scratch-python-c7a5abcd79bd>
2. <https://machinelearningmastery.com/implement-learning-vector-quantization-scratch-python/>

11) Useful Concepts

Learning Vector Quantization (or LVQ) is a type of Artificial Neural Network which also inspired by biological models of neural systems. It is based on prototype supervised learning classification algorithm and trained its network through a competitive learning algorithm similar to Self Organizing Map. It can also deal with the multiclass classification problem. LVQ has two layers, one is the Input layer and the other one is the Output layer. The architecture of the Learning Vector Quantization with the number of classes in an input data and n number of input features for any sample is given below:



How Learning Vector Quantization works?: Let's say that an input data of size (m, n) where m is the number of training examples and n is the number of features in each example and a label vector of size $(m, 1)$. First, it initializes the weights of size (n, c) from the first c number of training samples with different labels and should be discarded from all training samples. Here, c is the number of classes. Then iterate over the remaining input data, for each training example, it updates the winning vector (weight vector with the shortest distance (e.g Euclidean distance) from the training example).

The weight updation rule is given by:

```
if correctly_classified:
     $w_{ij}(\text{new}) = w_{ij}(\text{old}) + \alpha(t) * (x_i^k - w_{ij}(\text{old}))$ 
else:
     $w_{ij}(\text{new}) = w_{ij}(\text{old}) - \alpha(t) * (x_{ik} - w_{ij}(\text{old}))$ 
```

where α is a learning rate at time t , j denotes the winning vector, i denotes the i th feature of training example and k denotes the k th training example from the input data. After training the

LVQ network, trained weights are used for classifying new examples. A new example is labelled with the class of the winning vector.

Algorithm:

Step 1: Initialize reference vectors.

from a given set of training vectors, take the first "n" number of clusters training vectors and use them as weight vectors, the remaining vectors can be used for training.

Assign initial weights and classifications randomly

Step 2: Calculate Euclidean distance for $i=1$ to n and $j=1$ to m ,

$$D(j) = \sum (x_i - W_{ij})^2$$

find winning unit index j , where $D(j)$ is minimum

Step 3: Update weights on the winning unit w_i using the following conditions:

if $T = J$ then $w_i(\text{new}) = w_i(\text{old}) + \alpha[x - w_i(\text{old})]$

if $T \neq J$ then $w_i(\text{new}) = w_i(\text{old}) - \alpha[x - w_i(\text{old})]$

Step 4: Check for the stopping condition if false repeat the above steps.

12) Solved Lab Activities

<i>Sr.No</i>	<i>Allocated Time</i>	<i>Level of Complexity</i>	<i>CLO Mapping</i>
<i>1</i>	<i>45</i>	<i>Medium</i>	<i>CLO-3</i>

Activity 1: Implementation of LVQs in python

Solution:

```
import math

class LVQ :

    # Function here computes the winning vector

    # by Euclidean distance

    def winner( self, weights, sample ) :

        D0 = 0

        D1 = 0

        for i in range( len( sample ) ) :
```

```

        D0 = D0 + math.pow( ( sample[i] - weights[0][i] ), 2 )

        D1 = D1 + math.pow( ( sample[i] - weights[1][i] ), 2 )

    if D0 > D1 :

        return 0

    else :

        return 1

# Function here updates the winning vector

def update( self, weights, sample, J, alpha, actual ) :

    if actual == j:

        for i in range(len(weights)) :

            weights[J][i] = weights[J][i] + alpha * ( sample[i] - weights[J][i] )

    else:

        for i in range(len(weights)) :

            weights[J][i] = weights[J][i] - alpha * ( sample[i] - weights[J][i] )

# Driver code

def main() :

    # Training Samples ( m, n ) with their class vector

```

```
X = [[ 0, 0, 1, 1 ], [ 1, 0, 0, 0 ],  
      [ 0, 0, 0, 1 ], [ 0, 1, 1, 0 ],  
      [ 1, 1, 0, 0 ], [ 1, 1, 1, 0 ],]  
  
Y = [ 0, 1, 0, 1, 1, 1 ]  
  
m, n = len( X ), len( X[0] )  
  
# weight initialization ( n, c )  
  
weights = []  
  
weights.append( X.pop( 0 ) )  
  
weights.append( X.pop( 0 ) )  
  
  
# Samples used in weight initialization will  
  
# not use in training  
  
m = m - 2  
  
Y.pop(0)  
  
Y.pop(0)  
  
  
# training  
  
ob = LVQ()
```

```
epochs = 3

alpha = 0.1

for i in range( epochs ) :

    for j in range( m ) :

        # Sample selection

        T = X[j]

        # Compute winner

        J = ob.winner( weights, T )


        # Update weights

        ob.update( weights, T, J, alpha , Y[j])

# classify new input sample

T = [ 0, 0, 1, 0 ]

J = ob.winner( weights, T )

print( "Sample T belongs to class : ", J )

print( "Trained weights : ", weights )

if __name__ == "__main__":
```

`main()`

13) Graded Lab Tasks

The purpose of this lab task is to help students understand and implement LVQs in python.

Lab Task 1: Build and Train LVQs in Python

Download a dataset from Kaggle or UCI machine learning repository and train SOMs algorithm on the dataset.

Instructions for Submission:

4. Submit a Jupyter Notebook or Python script with the complete implementation.
5. Include comments in the code to explain each step.
6. Provide a brief report (1-2 pages) summarizing the following:
 - Data preprocessing steps
 - Model architecture and rationale for chosen hyperparameters
 - Training process and evaluation metrics

Lab 06

Objective:

The objective of this lab is to understand Keras – a commonly used python library for implementing artificial neural networks. In this lab, the students will learn how to implement ANNs in python using Keras library.

Activity Outcomes:

The activities provide hands - on practice with the following topics

- Implement ANN in Keras..
- Train ANN using a dataset.

Instructor Note:

As pre-lab activity, read and implement the following tutorials:

1. <https://www.educative.io/answers/how-to-build-a-simple-neural-network-using-keras>
2. <https://towardsdatascience.com/building-our-first-neural-network-in-keras-bdc8abbc17f5>

14) Useful Concepts

- **Keras:** Keras is a high-level, user-friendly API used for building and training neural networks. It is an open-source library built in Python that runs on top of TensorFlow. It was developed to enable fast experimentation and iteration, and it lowers the barrier to entry for working with deep learning. In this lab, we'll see how to install and start using Keras; the Sequential API; and the steps for building, compiling, and training a model.
- **Installing Keras:** To start with, you'll want to install TensorFlow. Depending on which operating system you're using, this might look slightly different, but for the most part, you can use pip, Python's package manager:

```
# First upgrade pip
pip install --upgrade pip

# Then install TensorFlow
pip install tensorflow
```

Once you've installed TensorFlow, all you need to do to use Keras is to run the following import statement at the top of your script or notebook:

```
from tensorflow import keras
```

- **Keras' Sequential API:** The Sequential API is the easiest way to use Keras to build a neural network. Using the Sequential class, it's possible to stack a variety of different layer types, one after the other, to produce a neural network. There are many types of layers available in the Keras Sequential API. One of the most common layer types is the Dense layer, a fully connected layer, but there are many others:
 - **Convolutional layer:** a layer for processing images — used for convolutional neural networks.
 - **Recurrent layer:** a layer for processing sequences of data — used for recurrent neural networks.
 - **MaxPooling layer:** a layer for down-sampling feature maps by taking the maximum value in non-overlapping rectangular blocks — used for preserving important features while reducing the chance of overfitting.
 - **Flatten layer:** a layer that flattens the multi-dimensional input tensors into a single dimension — used as a transition layer between convolutional or recurrent layers and fully connected layers in a neural network.

- **Dropout layer:** a layer that randomly sets input units to 0 (using a defined frequency) during training time — used as a regularization technique to prevent overfitting in neural networks.
- **Embedding layer:** a layer that represents words or phrases in a high-dimensional vector space — used to map words or phrases to dense vectors for use as input to a neural network.

These are just a few examples of the many types of layers available in the Keras Sequential API. Each layer is designed to perform a specific type of computation on the inputs, and they can be combined to create powerful neural network architectures.

- **Building a Model with Keras:** To build a model with the Keras Sequential API, the first step is to import the required class and instantiate a model using this class:

```
from tf.keras import Sequential

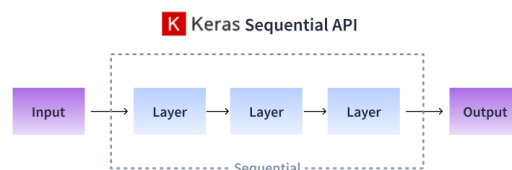
model = Sequential()
```

Next, choose the layer types you wish to include, and add them one at a time to the sequential model you've instantiated. For example, to add three Dense hidden layers and one output layer to an input layer, your code might look like this:

```
from tf.keras import Dense

model.add(Dense(input_shape=(16,)))
model.add(Dense(8))
model.add(Dense(4))
model.add(Dense(2))
model.add(Dense(1))
```

In this example, we've added an implicit input layer with 16 nodes and three hidden layers with 8, 4, and 2 nodes, respectively. Finally, we've added an output layer with just one node.



-
- **Compiling a Model with Keras:** Once you've built a model using the Keras Sequential API, you'll need to compile it before it can be used for training. When compiling the model, you'll need to specify both a loss function and an optimizer. There are many different options to choose from:

Loss functions:

- Mean Squared Error (MSE): a common loss function for regression problems — measures the average squared difference between the predicted and actual values.
- Binary Crossentropy: a loss function for binary classification problems — measures the cross-entropy between the predicted and actual binary distributions.
- Categorical Crossentropy: a loss function for multi-class classification problems — measures the cross-entropy between the predicted and actual categorical distributions.

Optimizers:

- Stochastic Gradient Descent (SGD): a simple optimization algorithm that updates the parameters by computing the gradient of the loss function with respect to the parameters.
- Adam: an optimization algorithm that adapts the learning rate based on the historical gradient information.
- RMSprop: an optimization algorithm that uses a moving average of squared gradients to normalize the gradient updates.

These are just a few examples of the many loss functions and optimizers available in Keras. The choice of loss function and optimizer will depend on the specific problem you're trying to solve and the characteristics of your data.

For example, to compile a model with Mean Squared Error as the loss function and Adam as the optimizer, we would use the following code:

```
model.compile(loss='mean_squared_error', optimizer='adam')
```

- **Fitting a Model with Keras:** Finally, to use Keras for deep learning, the compiled model must be fit to a training dataset. The training dataset should be prepared using a process that separates the independent variables, the features (or X variable) from the dependent variable, the target (or y variable). These variables should also be converted to NumPy arrays and reshaped as needed, and the `input_shape` parameter of the neural network will

need to be configured to match the training data shape. Once all of these preprocessing steps are in place, you can simply fit the model to the training data like so:

```
model.fit(x_train, y_train)
```

To evaluate the performance of the model after training, you can use evaluate:

```
test_loss, test_accuracy = model.evaluate(x_test, y_test)
```

15) Solved Lab Activites

<i>Sr.No</i>	<i>Allocated Time</i>	<i>Level of Complexity</i>	<i>CLO Mapping</i>
<i>1</i>	<i>30</i>	<i>Medium</i>	<i>CLO-3</i>
<i>2</i>	<i>30</i>	<i>High</i>	<i>CLO-3</i>

Activity 1:

Students will learn to implement an artificial neural network (ANN) using Keras on a dataset. They will conduct experiments to understand the impact of various hyperparameters and model architectures on the performance of the ANN.

Solution:

Building the Neural Network:

```
# Import necessary libraries
```

```
import numpy as np
```

```
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.layers import Dense, Flatten
```

```
from tensorflow.keras.utils import to_categorical
```

```
from tensorflow.keras.datasets import mnist
```

```
# Load and preprocess the dataset
```

```
(x_train, y_train), (x_test, y_test) = mnist.load_data()
```

```
x_train = x_train / 255.0
```

```
x_test = x_test / 255.0
```

```
# One-hot encode the labels

y_train = to_categorical(y_train)
y_test = to_categorical(y_test)


# Build the neural network model

model = Sequential([

    Flatten(input_shape=(28, 28)), # Flatten the input images to 1D vector

    Dense(128, activation='relu'), # First hidden layer with 128 neurons and ReLU activation

    Dense(64, activation='relu'), # Second hidden layer with 64 neurons and ReLU activation

    Dense(10, activation='softmax') # Output layer with 10 neurons (one for each class) and softmax
    activation

])


# Compile the model

model.compile(optimizer='adam',

              loss='categorical_crossentropy',

              metrics=['accuracy'])


# Train the model

model.fit(x_train, y_train, epochs=10, batch_size=32, validation_split=0.2)


# Evaluate the model

test_loss, test_acc = model.evaluate(x_test, y_test)
```

```
print(f'Test accuracy: {test_acc:.4f}')
```

Activity 2: Experiments

- **Experiment 1:** Change the number of neurons in the hidden layers and observe the effect on accuracy.

```
model = Sequential([  
    Flatten(input_shape=(28, 28)),  
    Dense(256, activation='relu'), # Changed to 256 neurons  
    Dense(128, activation='relu'), # Changed to 128 neurons  
    Dense(10, activation='softmax')  
])
```

- **Experiment 2:** Change the activation function to `sigmoid` and observe the effect on accuracy.

```
model = Sequential([  
    Flatten(input_shape=(28, 28)),  
    Dense(128, activation='sigmoid'), # Changed to sigmoid  
    Dense(64, activation='sigmoid'), # Changed to sigmoid  
    Dense(10, activation='softmax')  
])
```

Experiment 3: Change the optimizer to `sgd` and observe the effect on accuracy

```
model.compile(optimizer='sgd',  
              loss='categorical_crossentropy',  
              metrics=['accuracy'])
```

Experiment 4: Change the number of epochs to 20 and observe the effect on accuracy.

```
model.fit(x_train, y_train, epochs=20, batch_size=32, validation_split=0.2)
```

16) Graded Lab Tasks

The purpose of this lab task is to help students understand and implement artificial neural networks in Keras. The student will also learn how to perform experiments.

Lab Task 1: Building and Training a Neural Network Using Keras

Students will build and train a neural network to predict wine quality using the provided dataset. They will preprocess the data, create a neural network model, train it, and evaluate its performance. For this task, use the "Wine Quality" dataset from the UCI Machine Learning repository: [Wine Quality Data Set](#).

Instructions for Submission:

7. Submit a Jupyter Notebook or Python script with the complete implementation.
8. Include comments in the code to explain each step.
9. Provide a brief report (1-2 pages) summarizing the following:
 - Data preprocessing steps
 - Model architecture and rationale for chosen hyperparameters
 - Training process and evaluation metrics
 - Observations and insights from the results

Lab Task 2: Enhancing Neural Network Performance Using Keras

Students will demonstrate their ability to improve the performance of a neural network using Keras on a dataset from the Kaggle or UCI Machine Learning repository. They will learn to apply techniques such as hyperparameter tuning, regularization, and advanced optimizers to enhance model performance.

Instructions: Apply following improvement techniques

- Hyperparameter Tuning: Experiment with different numbers of neurons, layers, epochs, and batch sizes.
- Advanced Optimizers: Experiment with different optimizers like RMSprop or SGD with momentum.

Instructions for Submission:

1. Submit a Jupyter Notebook or Python script with the complete implementation.
2. Include comments in the code to explain each step.
3. Provide a brief report (1-2 pages) summarizing the following:
 - Initial model performance
 - Techniques applied for improvement
 - Improved model performance
 - Observations and insights from the results

Lab 07

Objective:

The objective of this lab is to learn the implementation of Convolutional Neural Network in Keras. A convolutional neural network (CNN or ConvNet) for short, is a class of deep neural networks, most commonly applied to image classification and computer vision problems.

Activity Outcomes:

The activities provide hands - on practice with the following topics

- Implement CNN in Keras
- Train CNN using a dataset in Keras

Instructor Note:

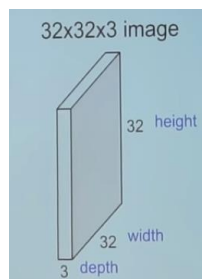
As pre-lab activity, read and implement the following tutorial:

1. <https://learnopencv.com/implementing-cnn-tensorflow-keras/>
2. <https://www.tensorflow.org/tutorials/images/cnn>

17) Useful Concepts

- **Convolutional Neural Network (CNN):** In deep learning, a convolutional neural network (CNN or ConvNet) is a class of deep neural networks. CNNs, like neural networks, are made up of neurons with learnable weights and biases. Each neuron receives several inputs, takes a weighted sum over them, pass it through an activation function and responds with an output. The whole network has a loss function and all the basics for neural networks will apply on CNNs as well. A convolutional neural network (CNN) consists of an input and an output layer, as well as multiple hidden layers. The hidden layers of a CNN typically consist of a series of convolutional layers that convolve with a multiplication or other dot product. The activation function is commonly a RELU layer, and is subsequently followed by additional convolutions such as pooling layers, fully connected layers and normalization layers, referred to as hidden layers because their inputs and outputs are masked by the activation function and final convolution. The final convolution, in turn, often involves backpropagation in order to more accurately weight the end product. CNNs have wide variety of applications in image and video recognition, recommender systems and natural language processing.

CNNs operate over images; Unlike neural networks, where the input is a vector, in CNNs the input is a multi-channelled image (3 channelled) as shown in the figure below:

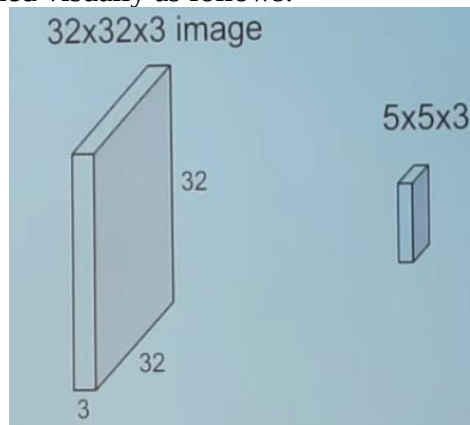


- **Basic terminology of CNN:** We should be aware of some basic terminology regarding CNN. These are as follows:-
 - **Filters:** CNN's make use of filters to detect what features, such as edges, are present throughout an image. A filter is just a matrix of values, called weights, that are trained to detect specific features. The filter moves over each part of the image to check if the feature it is meant to detect is present. To provide a value representing how confident it is that a specific feature is present, the filter carries out a convolution operation, which is an element-wise product and sum between two matrices. When the feature is present in part of an image, the convolution operation between the filter and that part of the image results in a real number with a high value. If the feature is not present, the resulting value is low.
 - **Convolutional:** When programming a CNN, each convolutional layer within a neural network should have the following attributes: Input is a tensor with shape (number of images) x (image width) x (image height) x (image depth). Convolutional kernels whose width and height are hyper-parameters, and whose depth must be equal to that of the image. Convolutional layers convolve the input and pass its result to the next layer.

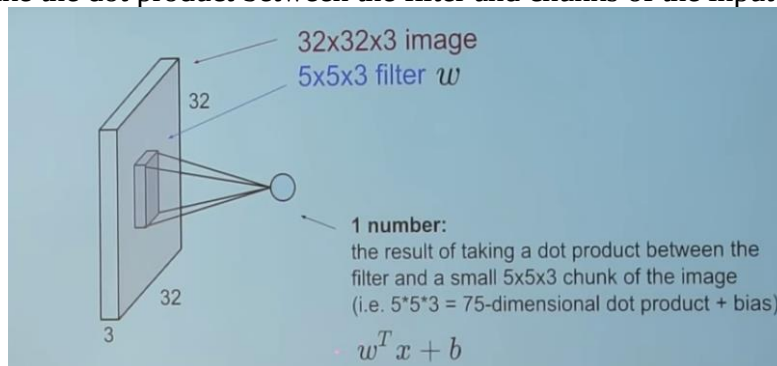
-
- **Pooling:** Convolutional networks may include local or global pooling layers to streamline the underlying computation. Pooling layers reduce the dimensions of the data by combining the outputs of neuron clusters at one layer into a single neuron in the next layer. Local pooling combines small clusters, typically 2×2 . Global pooling acts on all the neurons of the convolutional layer. In addition, pooling may compute a max or an average. Max pooling uses the maximum value from each of a cluster of neurons at the prior layer. Average pooling uses the average value from each of a cluster of neurons at the prior layer.
 - **Fully connected:** Fully connected layers connect every neuron in one layer to every neuron in another layer. It is in principle the same as the traditional multi-layer perceptron neural network (MLP). The flattened matrix goes through a fully connected layer to classify the images.
 - **Receptive field:** In neural networks, each neuron receives input from some number of locations in the previous layer. In a fully connected layer, each neuron receives input from every element of the previous layer. In a convolutional layer, neurons receive input from only a restricted subarea of the previous layer. Typically the subarea is of a square shape (e.g., size 5 by 5). The input area of a neuron is called its receptive field. So, in a fully connected layer, the receptive field is the entire previous layer. In a convolutional layer, the receptive area is smaller than the entire previous layer.
 - **Weights:** Each neuron in a neural network computes an output value by applying a specific function to the input values coming from the receptive field in the previous layer. The function that is applied to the input values is determined by a vector of weights and a bias (typically real numbers). Learning, in a neural network, progresses by making iterative adjustments to these biases and weights. The vector of weights and the bias are called filters and represent particular features of the input (e.g., a particular shape). A distinguishing feature of CNNs is that many neurons can share the same filter. This reduces memory footprint because a single bias and a single vector of weights are used across all receptive fields sharing that filter, as opposed to each receptive field having its own bias and vector weighting.
 - **Strides:** Strides is the number of pixels shifts over the input matrix. When the stride is 1 then we move the filters to 1 pixel at a time. When the stride is 2 then we move the filters to 2 pixels at a time and so on.
 - **Padding:** In CNNs, we can also apply a technique of filling zeros around the margin of the image to improve the sweep that is done with the sliding window. The parameter to define this technique is called “padding”. In CNNs, sometimes filter does not perfectly fit the input image. In this case, we have two options: We can apply a technique of filling zeros around the margin of the image so that it fits the input image perfectly. This is called zero-padding where we pad the gaps with zeros. Another option is to drop the part of the image where the filter does not fit the image perfectly. This is called valid padding which keeps only valid part of the image.

- **What is Convolution:** Convolutional neural networks are simple neural networks that use convolution in place of general matrix multiplication in at least one of their layers. The name convolutional neural network indicates that the network employs a mathematical operation called convolution. Convolution is a specialized kind of linear operation. In mathematics, convolution is a mathematical operation on two functions (f and g) that produces a third function expressing how the shape of one is modified by the other. The term **convolution** refers to both the result function and to the process of computing it. It is defined as the integral of the product of the two functions after one is reversed and shifted.

Convolution is explained visually as follows:

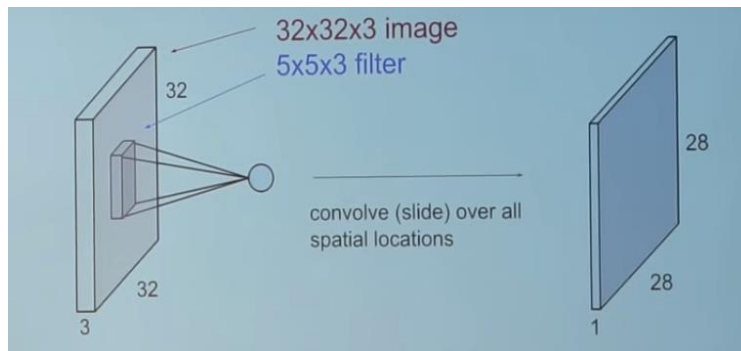


In the above plot, we take the 5x5x3 filter and slide it over the complete image and along the way take the dot product between the filter and chunks of the input image.



For every dot product taken, the result is a scalar.

Now, we convolve the complete image with the filter as shown below.



- **Basic building blocks of CNN:**

- A simple CNN is a sequence of layers, and every layer of a CNN transforms one volume of activations to another through a differentiable function.
- We use three main types of layers to build CNN architectures - Convolutional Layer, Pooling Layer, and Fully-Connected Layer. We will stack these layers to form a full CNN architecture.
- There are two other layers - ReLU Layer and Loss Layer that are also used to build a CNN. These layers are described below:
- Convolutional layer
 - The convolutional layer is the core building block of a CNN. The layer's parameters consist of a set of learnable filters (or kernels), which have a small receptive field, but extend through the full depth of the input volume.
 - During the forward pass, each filter is convolved across the width and height of the input volume, computing the dot product between the entries of the filter and the input and producing a 2-dimensional activation map of that filter.
 - As a result, the network learns filters that activate when it detects some specific type of feature at some spatial position in the input.
 - Stacking the activation maps for all filters along the depth dimension forms the full output volume of the convolution layer.
 - Every entry in the output volume can thus also be interpreted as an output of a neuron that looks at a small region in the input and shares parameters with neurons in the same activation map.
- Pooling layer
 - Another important concept of CNNs is pooling, which is a form of non-linear down-sampling. There are several non-linear functions to implement pooling among which max pooling is the most common. It partitions the input image into a set of non-overlapping rectangles and for each such sub-region, outputs the maximum.
 - Intuitively, the exact location of a feature is less important than its rough location relative to other features. This is the idea behind the use of pooling in convolutional neural networks.

-
- The pooling layer serves to progressively reduce the spatial size of the representation, to reduce the number of parameters, memory footprint and amount of computation in the network, and hence to also control overfitting. It is common to periodically insert a pooling layer between successive convolutional layers in a CNN architecture.
 - The pooling layer operates independently on every depth slice of the input and resizes it spatially. The most common form is a pooling layer with filters of size 2×2 applied with a stride of 2 downsamples at every depth slice in the input by 2 along both width and height, discarding 75% of the activations. In this case, every max operation is over 4 numbers. The depth dimension remains unchanged.
 - In addition to max pooling, pooling units can use other functions, such as average pooling or ℓ_2 -norm pooling.
 - ReLU layer
 - ReLU is the abbreviation of Rectified Linear Unit, which applies the non-saturating activation function.
 - It is given by the following formula $f(x) = \max(0, x)$.
 - It effectively removes negative values from an activation map by setting them to zero.
 - It increases the nonlinear properties of the decision function and of the overall network without affecting the receptive fields of the convolution layer.
 - ReLU is often preferred to other functions because it trains the neural network several times faster without a significant penalty to generalization accuracy.
 - Fully connected layer
 - Finally, after several convolutional and max pooling layers, the high-level reasoning in the neural network is done via fully connected layers.
 - Neurons in a fully connected layer have connections to all activations in the previous layer, as seen in regular (non-convolutional) artificial neural networks.
 - Their activations can thus be computed as an affine transformation, with matrix multiplication followed by a bias offset (vector addition of a learned or fixed bias term).
 - Loss layer

- The loss layer specifies how training penalizes the deviation between the predicted (output) and true labels and is normally the final layer of a neural network. Various loss functions are appropriate for different tasks may be used.
- Softmax loss is used for predicting a single class of K mutually exclusive classes.
- Sigmoid cross-entropy loss is used for predicting K independent probability values in [0,1].
- Euclidean loss is used for regressing to real-valued labels $(-\infty, \infty)$.

18) Solved Lab Activites

<i>Sr.No</i>	<i>Allocated Time</i>	<i>Level of Complexity</i>	<i>CLO Mapping</i>
1	15	Medium	CLO-4
2	15	Medium	CLO-4
3	15	Medium	CLO-4
4	15	Medium	CLO-4

Activity 1: Implementing Basic CNN model

Now, we will build a basic CNN model. We will stack several layers to build the model. We will create a second group of layers that will have 64 filters with a 5×5 window in the convolutional layer and a 2×2 window in the pooling layer. In this case, the number of input channels will take the value of the 32 features that we have obtained from the previous layer.

Solution:

```
model = models.Sequential()
model.add(layers.Conv2D(32, (5,5), activation = 'relu', input_shape=(28,28,1)))
model.add(layers.MaxPooling2D((2,2)))
model.add(layers.Conv2D(64, (5,5), activation = 'relu'))
model.add(layers.MaxPooling2D((2,2)))
model.summary()
```

Output:

```

Model: "sequential_2"
-----
Layer (type)                 Output Shape              Param #
-----
conv2d_2 (Conv2D)            (None, 24, 24, 32)       832
-----
max_pooling2d_2 (MaxPooling2 (None, 12, 12, 32)       0
-----
conv2d_3 (Conv2D)            (None, 8, 8, 64)         51264
-----
max_pooling2d_3 (MaxPooling2 (None, 4, 4, 64)         0
-----
Total params: 52,096
Trainable params: 52,096
Non-trainable params: 0
-----

```

Activity 2: Implementing Basic CNN model for classification

- In this activity, you need to add a densely connected layer to the above model for classification. we have 64 4x4 filters, is to add a densely connected layer, which will serve to feed a final layer of softmax activation function with the following code snippet: `model.add(layers.Dense(10, activation='softmax'))`
- In this example, we have to adjust the tensors to the input of the dense layer like the softmax, which is a 1D tensor, while the output of the previous one is a 3D tensor. So, we will first flatten the 3D tensor to one of 1D.
- Our output (4,4,64) must be flattened to a vector of (1024) before applying the Softmax with the following code snippet: `model.add(layers.Flatten())`

Solution:

```

model = models.Sequential()

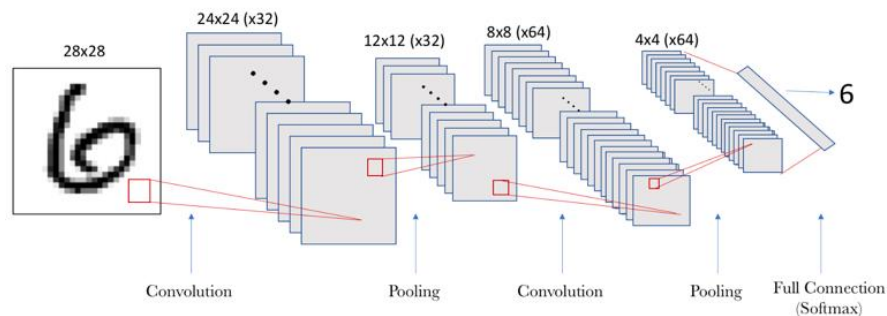
model.add(layers.Conv2D(32,(5,5),activation='relu', input_shape=(28,28,1)))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (5, 5), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Flatten())
model.add(layers.Dense(10, activation='softmax'))

```

Output:

Layer (type)	Output Shape	Param #
conv2d_4 (Conv2D)	(None, 24, 24, 32)	832
max_pooling2d_4 (MaxPooling2D)	(None, 12, 12, 32)	0
conv2d_5 (Conv2D)	(None, 8, 8, 64)	51264
max_pooling2d_5 (MaxPooling2D)	(None, 4, 4, 64)	0
flatten_1 (Flatten)	(None, 1024)	0
dense_1 (Dense)	(None, 10)	10250
Total params: 62,346		
Trainable params: 62,346		
Non-trainable params: 0		

A visual representation of the above information is shown in the following figure, where we see a graphic representation of the shape of the tensors that are passed between layers and their connections.



Activity 3: Training of the model

- Now that we have built our CNN model, its time to train it.
- It means to adjust the parameters of all the convolutional layers.
- Training can be done as follows:

Solution:

```
from keras.datasets import mnist
from keras.utils import to_categorical

(train_images, train_labels), (test_images, test_labels) = mnist.load_data()

train_images = train_images.reshape((60000, 28, 28, 1))
train_images = train_images.astype('float32') / 255

test_images = test_images.reshape((10000, 28, 28, 1))
```

```
test_images = test_images.astype('float32') / 255
```

```
train_labels = to_categorical(train_labels)
test_labels = to_categorical(test_labels)
```

```
batch_size = 100
epochs = 10
```

```
model.compile(loss='categorical_crossentropy',
              optimizer='sgd',
              metrics=['accuracy'])
```

```
model.fit(train_images, train_labels,
          batch_size=batch_size,
          epochs=epochs,
          verbose=1)
```

Output:

```
Downloading data from https://s3.amazonaws.com/img-datasets/mnist.npz
11493376/11490434 [=====] - 1s 0us/step
Epoch 1/10
60000/60000 [=====] - 31s 519us/step - loss: 0.9222 - accuracy: 0.7754
Epoch 2/10
60000/60000 [=====] - 31s 510us/step - loss: 0.2627 - accuracy: 0.9231
Epoch 3/10
60000/60000 [=====] - 31s 521us/step - loss: 0.1916 - accuracy: 0.9444
Epoch 4/10
60000/60000 [=====] - 31s 510us/step - loss: 0.1551 - accuracy: 0.9542
Epoch 5/10
60000/60000 [=====] - 31s 518us/step - loss: 0.1320 - accuracy: 0.9618
Epoch 6/10
60000/60000 [=====] - 30s 504us/step - loss: 0.1160 - accuracy: 0.9663
Epoch 7/10
60000/60000 [=====] - 31s 516us/step - loss: 0.1047 - accuracy: 0.9695
Epoch 8/10
60000/60000 [=====] - 30s 507us/step - loss: 0.0958 - accuracy: 0.9720
Epoch 9/10
60000/60000 [=====] - 31s 517us/step - loss: 0.0885 - accuracy: 0.9742
Epoch 10/10
60000/60000 [=====] - 32s 532us/step - loss: 0.0827 - accuracy: 0.9761
```

Activity 4: Evaluation of the model

- Now, its time to evaluate the model. Model evaluation can be done as follows

Solution:

```
test_loss, test_acc = model.evaluate(test_images, test_labels)
```

```
print('Test loss:', test_loss)
print('Test accuracy:', test_acc)
```

Output:

```
10000/10000 [=====] - 2s 209us/step
Test loss: 0.06909727230984718
Test accuracy: 0.980599994277954
```

19) Graded Lab Tasks

Lab Task 1: Building and Training a CNN Based Classification Model for MNIST Dataset. Experiment with various design choices such as number of layers, number of filters, activation functions, loss-functions.

Instructions for Submission:

10. Submit a Jupyter Notebook or Python script with the complete implementation.
11. Include comments in the code to explain each step.
12. Provide a brief report (1-2 pages) summarizing the following:
 - Data preprocessing steps
 - Model architecture and rationale for chosen hyperparameters
 - Training process and evaluation metrics
 - Observations and insights from the results

Lab Task 2: Enhancing Neural Network Performance Using Keras

Students will demonstrate their ability to improve the performance of a neural network using Keras on a dataset from the Kaggle or UCI Machine Learning repository. They will learn to apply techniques such as hyperparameter tuning, regularization, and advanced optimizers to enhance model performance.

Instructions: Apply following improvement techniques

- Hyperparameter Tuning: Experiment with different numbers of neurons, layers, epochs, and batch sizes.
- Advanced Optimizers: Experiment with different optimizers like RMSprop or SGD with momentum.

Instructions for Submission:

4. Submit a Jupyter Notebook or Python script with the complete implementation.
5. Include comments in the code to explain each step.
6. Provide a brief report (1-2 pages) summarizing the following:
 - Initial model performance
 - Techniques applied for improvement
 - Improved model performance
 - Observations and insights from the results

Lab 08

Objective:

The objective of this lab is to learn how to implement recurrent neural networks in keras.

Activity Outcomes:

The activities provide hands - on practice with the following topics

- Implement RNN model in Keras.
- Train RNN on a dataset in Keras.

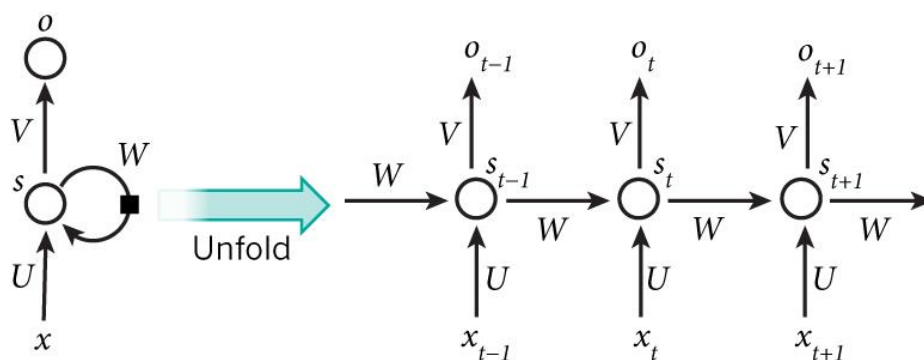
Instructor Note:

As pre-lab activity, read and implement the following tutorial:

1. <https://machinelearningmastery.com/understanding-simple-recurrent-neural-networks-in-keras/>
2. https://www.tensorflow.org/guide/keras/working_with_rnn

20) Useful Concepts

- **Recurrent Neural Networks (RNNs)** are a family of networks that are suitable for learning representations of sequential data like text in Natural Language Processing (NLP). The idea behind RNNs is to make use of sequential information. In a traditional neural network we assume that all inputs (and outputs) are independent of each other. But for many tasks that is a very bad idea. If we want to predict the next word in a sentence we better know which words came before it. RNNs are called recurrent because they perform the same task for every element of a sequence, with the output being depended on the previous computations. Another way to think about RNNs is that they have a “memory” which captures information about what has been calculated so far. In theory RNNs can make use of information in arbitrarily long sequences, but in practice they are limited to looking back only a few steps. Here is what a typical RNN looks like:



- The above diagram shows a RNN being unrolled (or unfolded) into a full network.
- By unrolling we simply mean that we write out the network for the complete sequence.
- For example, if the sequence we care about is a sentence of 5 words, the network would be unrolled into a 5-layer neural network, one layer for each word.
- **RNN Computations:**
 - We can describe the computations within an RNN in terms of equations.
 - The internal state of the RNN at a time t is given by the value of the hidden vector h_t , which is the sum of the product of the weight matrix W and the hidden state h_{t-1} at time $t-1$ and the product of the weight matrix U and the input x_t at time t , passed through the tanh nonlinearity.
 - The choice of tanh over other nonlinearities has to do with its second derivative decaying very slowly to zero.
 - This keeps the gradients in the linear region of the activation function and helps combat the vanishing gradient problem.

-
- The output vector y_t at time t is the product of the weight matrix V and the hidden state h_t , with softmax applied to the product so the resulting vector is a set of output probabilities:

$$h_t = \tanh(Wht - 1 + UXt)$$

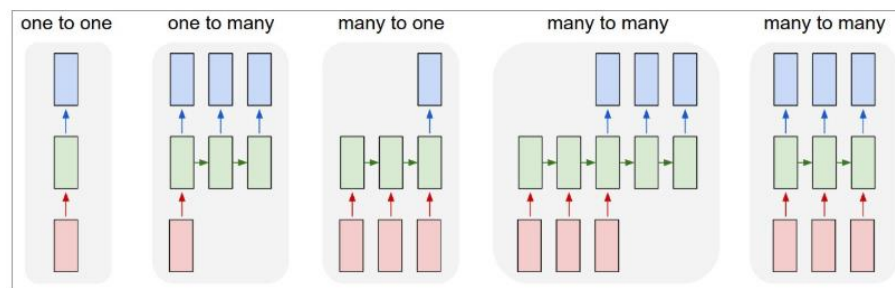
$$y_t = \text{softmax}(Vht)$$

- Keras provides the SimpleRNN (for more information refer to: <https://keras.io/layers/recurrent/>) recurrent layer that incorporates all the logic we have seen so far, as well as the more advanced variants such as LSTM and GRU that we will see later in this kernel.
- **SimpleRNN cells:**
 - Traditional multilayer perceptron neural networks make the assumption that all inputs are independent of each other. This assumption breaks down in the case of sequence data.
 - Time series data, such as stock prices, also exhibit a dependence on past data, called the secular trend.
 - RNN cells incorporate this dependence by having a hidden state, or memory, that holds the essence of the past.
 - The value of the hidden state at any point in time is a function of the value of the hidden state at the previous time step and the value of the input at the current time step, that is:

$$h_t = \phi(h_{t-1}, X_t)$$

- h_t and h_{t-1} are the values of the hidden states at the time steps t and $t-1$ respectively, and x_t is the value of the input at time t .
 - The above equation is recursive, that is, h_{t-1} can be represented in terms of h_{t-2} and x_{t-1} , and so on, until the beginning of the sequence.
 - This is how RNNs encode and incorporate information from arbitrarily long sequences.
- **RNN topologies**

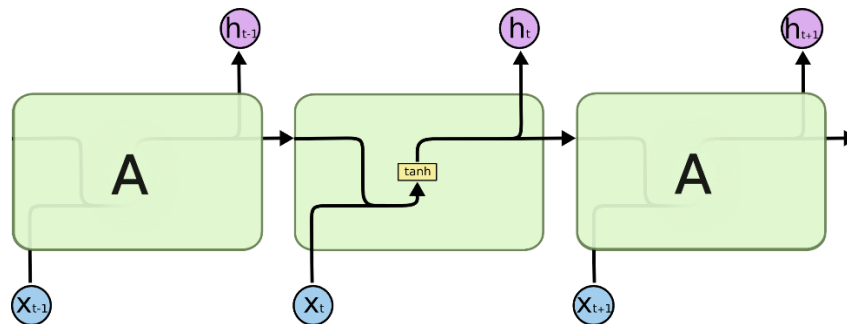
- The APIs for MLP and CNN architectures are limited. Both architectures accept a fixed-size tensor as input and produce a fixed-size tensor as output; and they perform the transformation from input to output in a fixed number of steps given by the number of layers in the model.
- RNNs don't have this limitation—you can have sequences in the input, the output, or both. This means that RNNs can be arranged in many ways to solve specific problems.
- RNNs combine the input vector with the previous state vector to produce a new state vector. This can be thought of as similar to running a program with some inputs and some internal variables.
- The RNNs are more exciting because they allow us to operate over sequences of vectors: Sequences in the input, the output, or in the most general case both.
- This property of being able to work with sequences gives rise to a number of common topologies shown below:



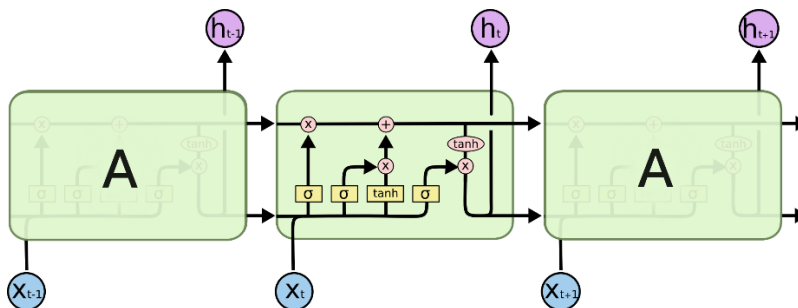
- In the above diagram each rectangle is a vector and arrows represent functions (e.g. matrix multiply).
- Input vectors are in red, output vectors are in blue and green vectors hold the RNN's state.
- From left to right:
 - Vanilla mode of processing without RNN, from fixed-sized input to fixed-sized output (e.g. image classification).
 - Sequence output (e.g. image captioning takes an image and outputs a sentence of words).
 - Sequence input (e.g. sentiment analysis where a given sentence is classified as expressing positive or negative sentiment).
 - Sequence input and sequence output (e.g. Machine Translation: an RNN reads a sentence in English and then outputs a sentence in French).
 - Synced sequence input and output (e.g. video classification where we wish to label each frame of the video).
- **Vanishing and exploding gradients**
 - Just like traditional neural networks, training the RNN also involves backpropagation.

-
- The difference in this case is that since the parameters are shared by all time steps, the gradient at each output depends not only on the current time step, but also on the previous ones.
 - This process is called backpropagation through time (BPTT).
 - Regular RNNs might have a difficulty in learning long range dependencies.
 - This kind of dependencies between sequence data is called long-term dependencies because the distance between the relevant information and the point where it is needed to make a prediction is very wide.
 - As this distance becomes wider, RNNs have a hard time learning these dependencies because it encounters either a vanishing or exploding gradient problem.
 - These problems arise during training of a deep network when the gradients are being propagated back in time all the way to the initial layer.
 - The gradients coming from the deeper layers have to go through continuous matrix multiplications because of the chain rule, and as they approach the earlier layers, if they have small values (<1), they shrink exponentially until they vanish and make it impossible for the model to learn. This is the vanishing gradient problem.
 - While on the other hand if they have large values (>1) they get larger and eventually blow up and crash the model. This is the exploding gradient problem.
 - While there are a few approaches to minimize the problem of vanishing gradients, such as proper initialization of the W matrix, using a ReLU instead of tanh layers, and pre-training the layers using unsupervised methods, the most popular solution is to use the LSTM or GRU architectures (discussed next).
 - These architectures have been designed to deal with the vanishing gradient problem and learn long term dependencies more effectively.
 - **Long Short Term Memory (LSTM)**

- **Long Short Term Memory** networks – usually just called **LSTMs** – are a special kind of RNN, capable of learning long-term dependencies. They work tremendously well on a large variety of problems, and are now widely used.
- LSTMs are explicitly designed to avoid the long-term dependency problem. Remembering information for long periods of time is practically their default behaviour.
- All recurrent neural networks have the form of a chain of repeating modules of neural network. In standard RNNs, this repeating module will have a very simple structure, such as a single tanh layer.
- The repeating module in a standard RNN contains a single layer as shown below:



- LSTMs also have this chain like structure, but the repeating module has a different structure.
- Instead of having a single neural network layer, there are four, interacting in a very special way as shown below:



21) Solved Lab Activities

<i>Sr.No</i>	<i>Allocated Time</i>	<i>Level of Complexity</i>	<i>CLO Mapping</i>
<i>1</i>	<i>60</i>	<i>Low</i>	<i>CLO-6</i>

Activity 1: Understand and implement the tutorial available at:

<https://machinelearningmastery.com/understanding-simple-recurrent-neural-networks-in-keras/>

22) Graded Lab Tasks

The purpose of this lab task is to help students understand and manipulate different data structures in Python, specifically lists, dictionaries, and tuples. Students will perform various operations on these data structures, including creation, modification, and data access.

Lab Task 1: Implement a Movie Review Classifier using Simple RNN in Keras

Students will build and train a simple RNN for classifying movie reviews. For this task, use IMDb movie reviews from Stanford AI Lab's Large Movie Review Dataset.

Instructions for Submission:

13. Submit a Jupyter Notebook or Python script with the complete implementation.
14. Include comments in the code to explain each step.
15. Provide a brief report (1-2 pages) summarizing the following:
 - Data preprocessing steps
 - Model architecture and rationale for chosen hyperparameters
 - Training process and evaluation metrics
 - Observations and insights from the results

Lab 09

Objective:

The objective of this lab is to understand how to implement long-short term memory in Keras.

Activity Outcomes:

The activities provide hands - on practice with the following topics

- Implement LSTM model in keras.
- Train LSTM using dataset in Keras.
-

Instructor Note:

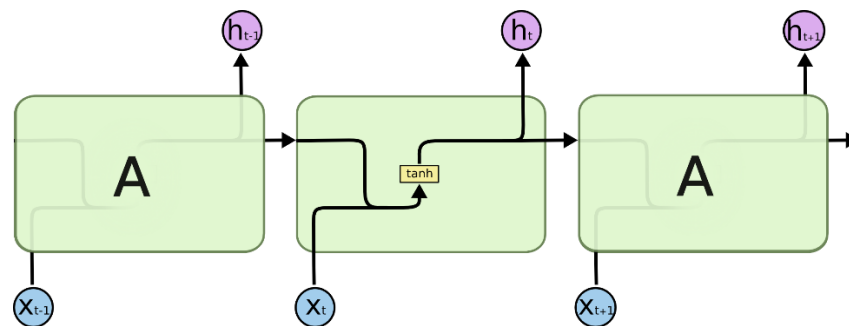
As pre-lab activity, read and implement the following tutorials:

1. <https://machinelearningmastery.com/time-series-prediction-lstm-recurrent-neural-networks-python-keras/>
2. https://keras.io/api/layers/recurrent_layers/lstm/

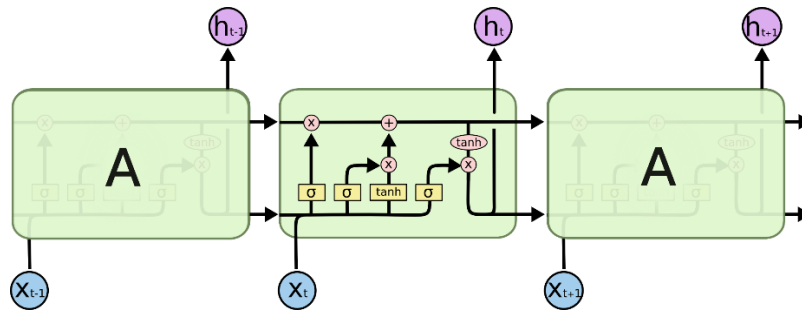
23) Useful Concepts

- **Long Short Term Memory (LSTM)**

- **Long Short Term Memory** networks – usually just called **LSTMs** – are a special kind of RNN, capable of learning long-term dependencies. They work tremendously well on a large variety of problems, and are now widely used.
- LSTMs are explicitly designed to avoid the long-term dependency problem. Remembering information for long periods of time is practically their default behaviour.
- All recurrent neural networks have the form of a chain of repeating modules of neural network. In standard RNNs, this repeating module will have a very simple structure, such as a single tanh layer.
- The repeating module in a standard RNN contains a single layer as shown below:



- LSTMs also have this chain like structure, but the repeating module has a different structure.
- Instead of having a single neural network layer, there are four, interacting in a very special way as shown below:



24) Solved Lab Activites

<i>Sr.No</i>	<i>Allocated Time</i>	<i>Level of Complexity</i>	<i>CLO Mapping</i>
<i>1</i>	<i>60</i>	<i>Low</i>	<i>CLO-6</i>

Activity 1: Understand and implement the tutorial available at:

<https://towardsdatascience.com/long-short-term-memory-lstm-in-keras-2b5749e953ac>

25) Graded Lab Tasks

The purpose of this lab task is to help students understand the implementation of LSTM in keras, and train the model using a dataset.

Lab Task: Build a LSTM model for sentiment classification

Students will build and train a LSTM for sentiment classification. For this task, use IMDb movie reviews from Stanford AI Lab's Large Movie Review Dataset.

Instructions for Submission:

16. Submit a Jupyter Notebook or Python script with the complete implementation.
17. Include comments in the code to explain each step.
18. Provide a brief report (1-2 pages) summarizing the following:
 - Data preprocessing steps
 - Model architecture and rationale for chosen hyperparameters
 - Training process and evaluation metrics
 - Observations and insights from the results